

آموزش صفر تا صد - نسخه ۲۰۲۶

Git و GitHub

برای دولوپرهای آماده‌ی تیم

از «کامیت و پوش بلام» تا حرفه‌ای شاخه، PR و کار در شرکت

Rebase & Conflict

Pull Request

GitHub Flow

Git 2.5x

GitHub Actions

Git Hooks

۱۶ فصل · تمرین با جواب · ۱۵ سناریوی نجات · ۲۴ سوال مصاحبه · چیت‌شیت و گلاساری

فهرست مطالب

نقشه دوره و روش مطالعه	شروع دوره	1
HEAD، SHA، سه ناحیه، snapshot	فصل ۱ – گیت چیست و چرا	2
SSH، clone، کلید، Git Bash، config، نصب ویندوز	فصل ۲ – نصب، پیکربندی و SSH	3
staging، add -p، git mv، Conventional Commits	فصل ۳ – سه ناحیه و کامیت بامعنا	4
dedup، index، git gc، آجکت‌ها، git، تور	فصل ۴ – زیر پوست گیت (git، تور)	5
pickaxe، pretty format، blame، فیلترها	فصل ۵ – git log حرفه‌ای	6
restore، reset، revert، reflog	فصل ۶ – برگرداندن‌ها بدون ترس	7
switch، merge، حل تعارض قدم‌به‌قدم	فصل ۷ – شاخه‌ها، Merge و تعارض	8
rebase -i، squash، قانون طلایی	فصل ۸ – Rebase و تاریخچه تمیز	9
semver، .gitignore، bisect و stash، tag	فصل ۹ – Stash، Tag و gitignore	10
upstream و fetch/pull، push -u، fork	فصل ۱۰ – ریموت‌ها، push/pull، فورک	11
squash merge، Rulesets، Issue، ریویو، PR، قالب	فصل ۱۱ – Pull Request، ریویو و Issue	12
GitHub Flow، Git Flow، trunk-based	فصل ۱۲ – گردش کارهای تیمی	13
۱۵ سناریوی «چه کار کنم اگر...»	فصل ۱۳ – کتاب نجات 📖	14
lint-staged و pre-commit، commit-msg، Husky	فصل ۱۴ – Git Hooks	15
amza، Checks، Dependabot، workflow، یامل	فصل ۱۵ – GitHub Actions و ابزارها	16
مرور سریع + واژه‌نامه + ۲۴ سوال مصاحبه	فصل ۱۶ – چیت‌شیت و مصاحبه	17

🎓 دوره جامع Git و GitHub – از «کامیت و پوش بلام» تا «آماده کار در شرکت»

نسخه دوره: ۲۰۲۶ – سازگار با Git 2.5x (با اشاره به تغییرات مهم Git 3.0 در راه) و GitHub فعلی (Rulesets، Fine-grained Token، GitHub Actions)
پیش‌نیاز: آشنایی مقدماتی با مفاهیم اولیه کامیت و پوش – این دوره از اصول پایه‌ای آغاز کرده و تمامی مهارت‌های پیشرفته مورد نیاز بازار کار را آموزش می‌دهد.

🎯 چرا این دوره طراحی شده و چه نیازی را پاسخ می‌دهد؟

بسیاری از برنامه‌نویسان در شروع کار، تنها با چند دستور ساده مثل commit و push کار می‌کنند و پروژه‌های خود را روی گیت‌هاب قرار می‌دهند. اما هنگام ورود به تیم‌ها و پروژه‌های بزرگ صنعتی با چالش‌های اساسی روبه‌رو می‌شوند:

- اگر بپرسند «rebase چیست و تفاوت عمیق آن با merge در کجاست؟» پاسخ آماده و دقیقی ندارند
- با **Merge Conflict** روبه‌رو نشده‌اند یا هنگام بروز تعارض، احساس سردرگمی می‌کنند
- روند اصولی ساخت **Pull Request**، کد ریویو (Code Review) و استراتژی‌های ادغام را عملی تجربه نکرده‌اند
- در محیط‌های شرکتی با جریان‌های کاری واقعی (Git Flow، Trunk-based، Protected Branches و CI) روبه‌رو می‌شوند و استانداردهای تیمی را نمی‌دانند

این دوره دقیقاً همین خلاء را پر می‌کند: ارتقای مهارت از یک ابزار ساده «ذخیره کد» به ابزار استاندارد و بین‌المللی «همکاری حرفه‌ای در تیم‌های نرم‌افزاری».

فصل	موضوع	چی یاد می‌گیری
فصل ۱	گیت چیست و چرا	مدل ذهنی درست: snapshot، سه ناحیه، HEAD، SHA
فصل ۲	نصب و اتصال به گیت‌هاب	نصب ویندوز، SSH key، config، Git Bash ، توکن
فصل ۳	سه ناحیه و کامیت بامعنا	git mv، diff، staging، کامیت اتمیک، Conventional Commits
فصل ۴	زیر پوست گیت	تور .git، آبجکت‌ها (blob/tree/commit)، git gc، index، dedup
فصل ۵	git log حرفه‌ای	فیلترها، blame، shortlog، pretty، pickaxe، main..feature
فصل ۶	برگرداندن‌ها	reflog، revert، reset، restore — بدون ترس!
فصل ۷	شاخه‌ها و Merge	branch واقعاً چیه، merge، حل تعارض قدم‌به‌قدم
فصل ۸	Rebase و تاریخچه تمیز	squash، interactive rebase، rebase، قانون طلایی
فصل ۹	Stash، Tag، gitignore	کمدهای کناری، نسخه‌ها، فایل‌های بی‌خیال، bisect
فصل ۱۰	گیت‌هاب و ریموت‌ها	fork، origin، push/pull/fetch، کلون و SSH
فصل ۱۱	Pull Request و ریویو	PR حرفه‌ای، Issue، Rulesets، merge strategies، code review
فصل ۱۲	گردش کار تیمی	trunk-based، Git Flow، GitHub Flow — کی کدوم؟
فصل ۱۳	کتاب نجات	«چه کار کنم اگر...» — ۱۵ سناریوی واقعی شرکت
فصل ۱۴	Git Hooks	Husky، pre-push، commit-msg، pre-commit، lint-staged
فصل ۱۵	GitHub Actions و ابزارها	CI ساده، Dependabot، Checks، امضای کامیت
فصل ۱۶	چیت‌شیت + گلاساری + مصاحبه	مرور سریع + ۲۴ سوال رایج مصاحبه با جواب

روش مطالعه (خیلی مهم)

این دوره سرشار از تمرین عملیه — برای هر فصل یک «زمین تمرین» داریم:

```
mkdir git-playground && cd git-playground
git init
```

یک پوشه آزمایشی بسازید و همه دستورات دوره را آنجا تمرین و اجرا کنید. اشتباه کردن در محیط آزمایشی کاملاً بی‌خطر و بهترین راه یادگیری است، اما خطا روی مخزن‌های اصلی پروژه هزینه دارد! پاسخ تمرین‌ها درون بخش‌های تاشو قرار داده

شده تا ابتدا خودتان راهکار را پیاده‌سازی کنید.

قانون طلایی دوره: گیت تقریباً هیچ‌وقت چیزی رو از بین نمی‌بره. تقریباً هر کار اشتباهی برگشت‌پذیره – فصل ۶ و ۱۳
ثابتش می‌کنن. پس ترست رو بذار کنار و تایپ کن.

پیش‌نیازهای فنی

ابزار	حداقل	چک کردن
Windows 10/11	–	–
Git for Windows	+2.4x (آخرین نصب کن)	<code>git --version</code>
حساب GitHub	رایگان	داری ☒
VS Code	توصیه‌شده	ادیتور پیش‌فرض ما

فصل ۱ – گیت چیست و چرا؟ (مدل ذهنی درست)

🎯 هدف این فصل: قبل از دستورات، مدل ذهنی درست رو بسازیم. ۹۰٪ استرس گیت از اینه که آدم نمی‌دونه «زیر hood چه خبره». آخر این فصل، کامیت رو دیگه «یک دکمه» نمی‌بینی؛ یک snapshot با شناسه می‌بینی – و همه چیز ساده می‌شه.

۱.۱ – مسئله‌ای که گیت حلش می‌کنه

همه برنامه‌نویسان در شروع کار این سناریو را تجربه کرده‌اند: ساخت فایل‌هایی مثل `↑inal-really-↑inal2.js`، `↑inal.js` و `final.js` و سردرگمی میان نسخه‌ها. گیت مانند «**Time Machine** کد» عمل می‌کند – هر کامیت یک عکس لحظه‌ای (Snapshot) پایدار از کل پروژه است که می‌توان به آن بازگشت، آن را با سایر نسخه‌ها مقایسه کرد و به صورت همزمان همراه سایر اعضای تیم روی آن کار کرد.

بدون VCS	با گیت
کپی پوشه با اسم تاریخ‌دار	هر کامیت = snapshot با نام و پیام
«کدوم نسخه جدیدتره؟»	تاریخچه کامل قابل جستجو
همکاری = فایل جابجایی با فلش!	merge و branch و PR
«دیروز کار می‌کرد، الان خرابه!»	diff دقیق بین هر دو لحظه
اشتباه = از دست رفتن کد	تقریباً همه چیز قابل برگشت

VCS ها سه نسل داشتن – فهمیدنش کمک می‌کنه بفهمی گیت چرا این‌شکلیه:



گیت توزیع‌شده (Distributed) است: کل تاریخچه پروژه به صورت محلی روی سیستم هر توسعه‌دهنده قرار دارد. بدون اینترنت هم می‌توان کامیت زد، branch ساخت و تاریخچه را دید. گیت‌هاب فقط «فضای اشتراک و ریموت» است – گیت‌هاب ≠ گیت! (فصل ۱۰ به تفصیل بررسی می‌شود).

💡 گیت رو Linus Torvalds (خالق لینوکس) در ۲۰۰۵ ساخت – اول برای مدیریت کد خود لینوکس. اسمش به معنی «آدم مزاحم/زورگو» به انگلیسی عامیانه است – شوخی خودش. امروز استاندارد بزرگ صنعت است و Git 2.55 آخرین نسخه پایدارش است (و Git 3.0 در راه است که SHA-256 رو پیش فرض می‌کنه – برای کار روزمره‌ها هیچ فرقی نمی‌کنه).

۱.۲ – مهم‌ترین حقیقت: کامیت = Snapshot، نه Diff

این جمله رو قاب کن: «گیت به snapshot فکر می‌کنه، نه تغییرات.»

هر کامیت یک عکس کامل از کل پروژه در اون لحظه است (به صورت بهینه: فایل‌های تغییر یافته جدید ذخیره می‌شن، بقیه به snapshot قبلی اشاره می‌کنن). نتیجه جادویی:

(یک کامیت C هر) C1 ← C2 ← C3 ← C4

هر کامیت: - یک SHA یکتا دارد: `a1b2c3d...` (مثل اثر انگشت از محتوا – هش SHA-1) - به کامیت قبل از خودش اشاره می‌کنه (پدر) - دارد: author، تاریخ، و پیام

یعنی تاریخچه یک زنجیره است. هر چیزی که بهش SHA بدی، همون لحظه رو exact برمی‌گردونی. این پایه «برگشت‌ناپذیر نبودن ترس‌ها» است: کامیت زده‌ای؟ برای همیشه اونجاست، حتی اگه بترونیش (فصل ۶: reflog).

۱.۳ – سه ناحیه: جایی که ۹۹٪ سردرگمی‌ها شروع می‌شه

وقتی کار می‌کنی، کدت در یکی از این سه ناحیه است:



ناحیه	چیه	شبیه به
Working Directory	فایل‌هایی که الان ویرایش می‌کنی	میز کارت
Staging Area	«چه فایل‌هایی در کامیت بعدی برن» - انتخاب دستی تو	سبد خرید فروشگاه 🛒
Repository	کامیت‌های ثبت‌شده، داخل پوشه مخفی <code>.git</code>	انبار امن

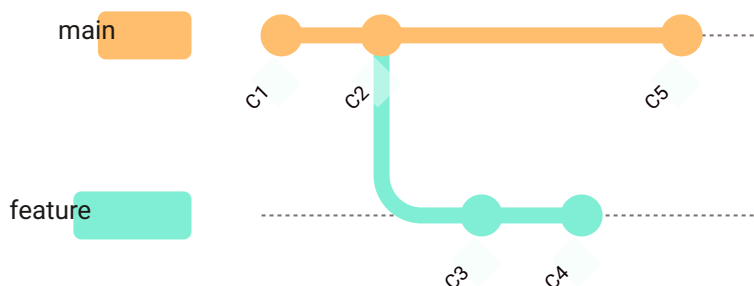
چرا staging جدا داریم؟ چون می‌خواهیم دقیقاً انتخاب کنی چه چیزی توی کامیت بره. مثلاً دو باگ رو فیکس کردی ولی می‌خواهی هر فیکس کامیت جدا بشه (کامیت اتمیک – فصل ۳): فقط فایل‌های مربوط به باگ اول رو add می‌کنی، کامیت می‌زنی، بعد بقیه.

قانون: `git add` = «بذار توی سبد». `git commit` = «سبد رو ثبت کن». فایلی که add نشده، در کامیت نیست – هرچقدر هم ویرایشش کردی!

۱.۴ – HEAD و Branch: اشاره‌گرها، نه فایل‌ها

مفهومی که وقتی بفهمیش، شاخه‌ها برایت معما نمی‌شن:

- **Commit** = یک snapshot با SHA
- **Branch** = فقط یک اسم قابل جابجایی که به یک کامیت اشاره می‌کنه – بس! یک فایل متنی ۴۱ بایتی در `.git/refs/heads/`
- **HEAD** = اشاره‌گر به «الان کجای تاریخچه‌ای» – معمولاً به آخرین کامیت شاخه فعلی



در نمودار بالا: `main` به C5 اشاره می‌کنه و `feature` به C4. «ساختن شاخه» یعنی ساختن یک اشاره‌گر جدید – به همین سادگی و سریعی (نه کپی کل پروژه!). «کامیت زدن» یعنی snapshot جدید + جابجا شدن اشاره‌گر شاخه فعلی به آن.

چکشت رو بردار و خودت ببین (در یک پروژه گیت‌دار):

```
cat .git/HEAD
# ref: refs/heads/main      ← اشاره می‌کند به main به شاخه HEAD
cat .git/refs/heads/main
# a1b2c3d4e5f6...          ← main = فقط یک SHA!
```

وقتی HEAD مستقیم به یک کامیت اشاره کنه (نه به شاخه)، در حالت **Detached HEAD** هستی – فصل ۱۳ توضیح می‌ده چطوری بی‌دردسر خارج شی.

۱.۵ – چرا SHA ها مهم‌اند؟

هر کامیت یک SHA کوتاه‌شده دارد مثل `a1b2c3d` . کجاها می‌بینی‌ش:

- `git log` – تاریخچه
- `git show a1b2c3d` – نمایش یک کامیت خاص
- `git checkout a1b2c3d` – سفر در زمان به آن لحظه
- `git diff a1b2c3d..HEAD` – مقایسه دو لحظه

چون SHA از **محتوا** محاسبه می‌شه، اگه کسی تاریخچه رو دستکاری کنه SHA ها عوض می‌شن و دیده می‌شه – یعنی گیت به صورت توکار **تخم‌مرغ‌زنی تاریخچه** رو لو می‌ده. (پایه امنیت گیت و پایه بحث «`rebase` = بازنویسی تاریخچه» در فصل ۸.)

۱.۶ – گیت در مقابل گیت‌هاب (و بقیه)

جدولی که خیلی‌ها قاطی می‌کنن:

GitHub	Git	
سرویس ابری میزبانی مخزن‌های گیت	نرم‌افزار کنترل نسخه (روی سیستم تو نصب می‌شه)	چی‌ه؟
ابر – فقط با اینترنت	لوکال، آفلاین هم کار می‌کنه	کجا؟
GitHub، GitLab، Bitbucket	Git، (Mercurial قدیمی)	نمونه‌ها
پلتفرم ابری اشتراک کد، میزبانی مخزن، PR ها و CI/CD	ابزار روزمره کنترل نسخه در ترمینال یا ادیتور	نقش برای توسعه‌دهنده

📌 گیت‌هاب فقط یک میزبان است. همه‌چیزی که در این دوره درباره گیت می‌آموزی (`branch`, `merge`, `rebase`...) در هر شرکت – حتی اونهایی که GitLab یا Bitbucket دارند – یکسانه. فقط PR و CI گیت‌هاب-خاص‌اند.

جمع‌بندی فصل ✓

- گیت = VCS توزیع‌شده؛ کل تاریخچه لوکال است؛ گیت‌هاب فقط میزبان ابری است
- کامیت = **snapshot کامل** با SHA یکتا و اشاره به پدر – زنجیره تاریخچه
- سه ناحیه: Working (میز کار) → Staging → `add` (سبد خرید) → Repository → `commit` (انبار)

- **Branch** = اشاره‌گر به یک کامیت (فایل چندبایتی!) – نه کپی پروژه
- **HEAD** = «الان کجایی»؛ سفر در زمان با SHA ممکنه
- گیت تاریخچه را hash می‌کند – دستکاری لو می‌رود

تمرین فصل ۱

1. سه ناحیه گیت را با analogi سبد خرید توضیح بده: **add** چه می‌کند؟ **commit** چه می‌کند؟
2. در یکی از پروژه‌های خودت (که گیت دارد)، **cat .git/HEAD** و سپس محتوای فایل **refs** را ببین. اسم شاخه‌ات چیه؟ SHA آخرین کامیت؟
3. با دستور **git log --oneline** ده کامیت آخر یکی از پروژه‌های قبلی خود را مرور کنید. (این تمرین صرفاً برای درک شکل خروجی است؛ از فصل‌های بعد یاد می‌گیرید تاریخچه‌ای استاندارد بسازید).
4. سوال مفهومی: چرا ساختن ۱۰ شاخه در گیت سریع و بی‌هزینه است، ولی در ذهن خیلی‌ها «سنگین» تصور می‌شود؟

▼ جواب‌ها

1. **add** = گذاشتن کالا در سبد (انتخاب دقیق چیزی که ثبت شود)؛ **commit** = پرداخت و ثبت نهایی سبد در انبار؛ آنچه در سبد نگذاشتی، ثبت نمی‌شود.
2. **HEAD** معمولاً **ref: refs/heads/main** است و فایل **refs** فقط یک ۴۰ SHA کاراکتری دارد – یعنی شاخه واقعاً فقط یک اشاره‌گر است.
3. چون شاخه فقط یک فایل ۴۱ بایتی است؛ «سنگین بودن» فقط تصور ذهنی است – یکی از هدف‌های این دوره رفع همین تصور است.

➔ **فصل بعد:** نصب، پیکربندی حرفه‌ای و اتصال امن به گیت‌هاب با SSH.

فصل ۲ – نصب، پیکربندی و اتصال امن به گیت‌هاب

🎯 هدف: نصب تمیز روی ویندوز، config هایی که یک بار برای همیشه می‌زنی، و اتصال به گیت‌هاب با SSH (دیگه رمز و توکن تایپ نکنی). آخر فصل یک پروژه واقعی clone می‌کنی.

۲.۱ – نصب Git روی ویندوز

۱. برو به <https://git-scm.com/download/win> → آخرین نسخه (Git 2.5x) رو دانلود کن

۲. موقع نصب، این گزینه‌ها مهمان (بقیه پیش‌فرض):

صفحه	گزینه	چرا
Select Components	پیش‌فرض (Git Bash + Git GUI تیک‌خورده)	Git Bash ترمینال ماست
Choosing the default editor	Use Visual Studio Code as Git's default editor ★	پیام‌های کامیت و rebase با VS Code باز می‌شن – راحت‌ترین
Adjusting your PATH	Git from the command line and also from 3rd-party software (پیش‌فرض)	git در همه ترمینال‌ها در دسترس
HTTPS transport	OpenSSL (پیش‌فرض)	—
Line ending conversions	Checkout Windows-style, commit Unix-style ★	پایین‌تر کامل توضیح می‌دم – مهم!
Terminal emulator	Use MinTTY (پیش‌فرض)	Git Bash بهتر از cmd
Experimental options	تیک نزن	—

۳. تست:

```
git --version
# git version 2.5x.windows.1
```

⚠️ داستان CRLF/LF – دردی که هر دولوپر ویندوزی یک بار می‌کشه

ویندوز خط جدید را با CRLF ثبت می‌کند، لینوکس/مک با LF. اگر تنظیم نباشه، diff های الکی پر از «کل فایل تغییر کرده» می‌شن و تیم‌های مختلط (ویندوزی+مکی) دیوانه می‌شن. راه‌حل دو لایه:

- موقع نصب: گزینه « `core.autocrlf=true` » = « Checkout Windows-style, commit Unix-style » – موقع checkout به CRLF تبدیل کن، موقع commit همیشه LF ذخیره کن
- بهتر از اون (روش مدرن ★): در ریشه هر پروژه فایل `.gitattributes` بذار که برای همه تیم یکسان باشه:

```
# .gitattributes
* text=auto eol=lf      # ذخیره شو LF همه فایل‌های متنی: در مخزن
*.png binary            # باینری‌ها دست نزن
```

۲.۲ – پیش‌گرم: دستورات پرکاربرد Git Bash

گیت یک ابزار خط فرمانه – پس خط فرمان هم باید دستت بیاد. Git Bash (که با گیت نصب شد) یک محیط لینوکسی روی ویندوزه؛ این‌ها صد درصد برای کار روزانه کافی‌ان:

```
pwd                # (print working directory) الان کجام؟
ls -la             # (پوشه) حتی مخفی‌ها -a لیست با جزئیات؛
cd my-app          # خانه ~ cd یک پله بالا؛ .. cd برو به پوشه؛

mkdir docs         # پوشه بساز
touch notes.txt    # فایل خالی بساز
cp app.js app.bak.js # کپی
mv app.js src/     # جابه‌جایی (و تغییر نام!)
rm notes.txt       # حذف فایل
rm -rf old-folder/ # حذف پوشه - ⚠ بدون سطل بازیافت!

cat README.md      # محتوای فایل را نمایش بده
echo "hi" > file.txt # خروجی را در فایل بریز (< بازنویسی؛ >> افزودن)
grep -rn "TODO" src/ # (شماره خط=n، بازگشتی=r) جست‌وجوی رشته در فایل‌ها
chmod +x script.sh  # قابل اجرا کردن فایل (فصل ۱۴ بهش نیاز داری)
```

مهارت‌های کوچکی که سرعتت رو چند برابر می‌کنن:

- **Tab** = تکمیل خودکار اسم فایل/پوشه – تایپ طولانی ممنوع
- **Ctrl+C** = قطع دستور گیرکرده؛ **clear** = پاک کردن صفحه
- `..` یعنی همین پوشه، `..` پوشه والد، `~` یعنی `C:\Users\YourUsername`
- زنجیره‌سازی: `git log --oneline | grep "feat"` – خروجی یک دستور را به دستور بعدی بده (پایپ |)

💡 همین دستورات مبنای هوک‌ها و اسکریپت‌های فصل ۱۴ اند (`grep`، `chmod`، شل). ده دقیقه امروز، ساعتها راحتی بعداً.

۲.۳ – پیکربندی: یک بار برای همیشه

گیت سه سطح config دارد: `system` (کل ویندوز) < `global` (کاربر تو ★) < `local` (فقط این مخزن).

```
# هویت تو - با همون ایمیلی که اکانت گیت‌هابت ثبت شده! (کامیت‌ها به پروفایلت وصل می‌شن)
git config --global user.name "Your Name"
git config --global user.email "you@example.com"

# شاخه پیش‌فرض جدیدها
git config --global init.defaultBranch main

# ادیتور و ابزار
git config --global core.editor "code --wait"

# های حرفه‌ای - یک بار بزن، عمراً پشیمون نشو alias پرکاربردترین
git config --global alias.st "status -sb"
git config --global alias.lg "log --oneline --graph --decorate --all"
git config --global alias.last "log -1 HEAD --stat"
git config --global alias.unstage "restore --staged"

# هست upstream برای شاخه‌ای که برایش push پیش‌فرض
git config --global push.autoSetRemote true

# (برای HTTPS) رمز عبور/توکن رو ویندوز خودش نگه داره
git config --global credential.helper manager
```

بعدش این رو بزن و نتیجه رو ببین:

```
git config --global --list
```

💡 از این به بعد دستور پرتکرارت خواهد بود - گراف تاریخیچه با یک جفت حرف. تو VS Code هم افزونه **GitLens** رو نصب کن؛ تاریخیچه‌خوانی فوق حرفه‌ای داخل ادیتور.

۲.۴ - اتصال به گیت‌هاب: SSH (روش استاندارد شرکت‌ها) ★

دو راه احراز هویت برای push/pull داریم:

روش	چیست	توصیه استاندارد
HTTPS + Personal Access Token	رمز یکبارمصرف به جای پسورد	برای کارهای موردی
SSH key	جفت‌کلید رمزنگاری؛ یک بار تنظیم، همیشه بدون رمز	★ استاندارد؛ روی همه شرکت‌ها رایجه

ساخت کلید SSH (ویندوز، در Git Bash)

```
# ۱. ساخت جفت کلید ed25519 (استاندارد مدرن)
ssh-keygen -t ed25519 -C "you@example.com"
# Enter (می‌تونی خالی بذاری) passphrase بزن (مسیر پیش فرض) و یک Enter بزن

# ۲. کلید عمومی رو نمایش بده و کپی کن
cat ~/.ssh/id_ed25519.pub
# ssh-ed25519 AAAAC3NzaC...xyz you@example.com
```

🔑 دو کلید ساخته شد: `id_ed25519` (خصوصی – هرگز جایی نفرست/آپلود نکن!) و `id_ed25519.pub` (عمومی – همینکه به گیت‌هاب می‌دی).

اضافه کردن به گیت‌هاب

1. GitHub → Settings → SSH and GPG keys → New SSH key → پروفایل
2. Title: اسم کامپیوترت (مثلاً `Work-Laptop` یا `Key`), `Personal-PC`: محتوای `.pub` رو paste کن
3. تست:

```
ssh -T git@github.com
# Hi <username>! You've successfully authenticated... 🎉
```

و توکن (برای وقتی HTTPS خواستی)

GitHub → Settings → Developer settings → Personal access tokens → Fine-grained tokens → Generate: فقط دسترسی به مخزن‌های مشخص، فقط دسترسی‌های لازم (Contents: Read and write)، تاریخ انقضا مشخص. توکن مثل پسورده – فقط یک بار نمایش داده می‌شه، نگهش دار.

🔔 اگر روزی کلید خصوصی یا توکن لو رفت: فوراً از گیت‌هاب حذفش کن (Revoke) و یکی جدید بساز. (و اگر در کد آپلود شده بود، فصل ۱۳ سناریوی «فایل حساس در کامیت» رو ببین.)

۲.۵ – اولین مخزن: clone و init

دو راه شروع یک پروژه:

الف) از صفر – `init`

```
mkdir my-app && cd my-app
git init
# Initialized empty Git repository in ../my-app/.git/
```

پوشه `.git` ساخته شد – مغز کل تاریخچه. هیچ وقت داخلش دستکاری نکن (فقط نگاه کن!).

ب) از گیت‌هاب – `clone` (پرکاربردتر در شرکت!)

```
# از URL روی گیت‌هاب داری (Code → SSH دکمه سبز):
git clone git@github.com:Username/my-project.git
cd my-project
git lg # ای که زدیم – تاریخچه رو ببین alias با
```

جدول کارهای روز اول در هر شرکت جدید (این رو حفظ کن – دقیقاً همین کار رو می‌کنی):

```
# ۱. کن add بده تا به مخزن IT دسترسی بگیر: ایمیل شرکت رو به تیم
# ۲. سیستم رو به اکانت گیت‌هابت اضافه کن SSH کلید
# ۳. کلون کن:
git clone git@github.com:company-name/project.git
# ۴. چه خبره و برنج محیطی بزن main ببین
cd project && git lg
npm install # یا هر نصبی پروژه لازم داره
```

۲.۶ – معرفی مخزن آزمایشی دوره: `git-playground`

از این فصل به بعد، همه تمرین‌ها در این زمین امن انجام می‌شه:

```
mkdir git-playground && cd git-playground
git init
echo "# زمین تمرین" > README.md
git add README.md
git commit -m "chore: init playground"
```

حالا یک فایل بساز و با `git status` هر لحظه ببین کجای سه‌ناحیه است – این «دیدن» عادتی که تا آخر دوره باهاته:

```
echo "v1" > app.js
git st
# ?? app.js ← Untracked: گیت هنوز نمی‌شناسدش
git add app.js
git st
# A app.js ← Added: در سبد کامیت بعدی
git commit -m "feat: add app v1"
git st
# nothing to commit, working tree clean ✓
```

✓ جمع‌بندی فصل

- Git Bash = ترمینال لینوکسی تو: Tab + `ls/cd/mv/rm/grep/chmod` و ↑ - پایه هوک‌های فصل ۱۴
- Git for Windows نصب کردی؛ ادیتور پیش‌فرض VS Code؛ توجه به `.gitattributes` + `autocrlf` (CRLF/LF)
- config ها یک‌بار برای همیشه: alias ، `init.defaultBranch main` ، `user.name/email` های `st` و `lg`
- SSH: جفت‌کلید ed25519؛ `.pub` به گیت‌هاب، کلید خصوصی محرمانه؛ `ssh -T git@github.com` تست اتصال
- `init` برای پروژه نو، `clone` برای ورود به پروژه موجود - روز اول شرکت = `clone` + آشنایی با `git lg`
- زمین تمرین `git-playground` آماده شد

📝 تمرین فصل ۲

1. هر سه alias این فصل رو بزن و `git lg` را روی یکی از مخزن‌های لوکال خود اجرا کنيد.
2. کلید SSH بساز و به گیت‌هاب وصل کن (اگه از قبل داری، فقط `ssh -T` بزن و اتصال رو تأیید کن). چند کلید داری؟
`ls ~/.ssh/*.pub`
3. زمین تمرین رو بساز و چرخه کامل `untracked` → `staged` → `committed` رو با `git st` دنبال کن.
4. به یکی از مخزن‌های خودت `.gitattributes` با محتوای فصل اضافه کن.
5. در Git Bash: با `grep -rn "TODO"` بگرد ببین چند TODO در پروژه‌ات مونده؛ بعد `cd ~` و `ls` - اینجا کجاست؟

▼ نکته تمرین ۲

```
ls ~/.ssh/*.pub
ssh -T git@github.com
```

اگه خروجی `Hi <username>!` بود، همه‌چیز آماده است. چند کلید داشتن اشکالی نداره (هر دستگاه یکی)، ولی لیستش رو در گیت‌هاب گاهی مرور کن و کلید دستگاه‌های قدیمی رو حذف کن.

➔ **فصل بعد:** هنر کامیت بامعنا - چیزی که تو تیم‌ها واقعاً بهش نمره می‌دن.

فصل ۳ – سه ناحیه در عمل و هنر کامیت بامعنا

🎯 هدف: تسلط کامل بر staging (شامل انتخاب تکه‌تکه‌ای فایل)، خواندن diff و log مثل حرفه‌ای‌ها، و یادگیری استاندارد Conventional Commits – همون چیزی که در ریویو تیم به چشم می‌آید.

۳.۱ – status و log و diff : چشم‌های تو

این سه دستور، «فقط نگاه کردن» اند و بی‌خطرترین ابزارهای تو. عادت کن قبل و بعد از هر کاری صدا بزنی.

```
git status          # خلاصه و قشنگ (-sb دستور) کجای سه ناحیه‌ام؟
git log --oneline   # تاریخچه فشرده
git lg              # گراف همه شاخه‌ها (فصل ۲ alias)
git diff            # Working ↔ Staging: چه ویرایشی هنوز نشده؟ add
git diff --staged   # Staging ↔ Commit: چه چیزی «قراره» کامیت بشه؟
git diff HEAD       # جمع کل: کارم از آخرین کامیت چه فرقی کرده؟
git show a1b2c3d    # محتوای یک کامیت خاص
```

💡 قاعده ذهنی ساده: diff بدون flag = «هنوز به سبد نگذاشتم»؛ diff --staged = «توی سبد، هنوز ثبت نشده». یادت باشه کامیت بدون add، خالیه. (و log فقط شروع ماجراست – نسخه حرفه‌ای‌اش کل فصل ۵ است.)

خواندن خروجی diff

```
diff --git a/app.js b/app.js
index 83db48f..bf2b9f4 100644
--- a/app.js
+++ b/app.js
@@ -1,4 +1,5 @@
  const express = require('express');
+const helmet = require('helmet');           // خط اضافه‌شده (سبز)
  const app = express();
- app.use(cors());                           // خط حذف‌شده (قرمز)
+ app.use(cors({ origin: 'https://myshop.com' }));
```

@@ -1,4 +1,5 @@ یعنی «از خط ۱ نسخه قبلی ۴ خط بود؛ در نسخه جدید ۵ خط شد».

۳.۲ – تغییر و جابه‌جایی فایل‌ها: mv و rm زیر نظر گیت

جابه‌جایی و حذف، دو عملی‌ان که «بی‌اجازه گیت» کار رو شلوغ می‌کنن. راه تمیز:

```
git mv app.js src/app.js      # staging جابه‌جایی/تغییر نام + ثبت در
git rm old-notes.md          # (کامیت بعدی حذف را ثبت می‌کند) index حذف فایل از دیسک و از

# معادل دستی (همین نتیجه، دو دستور):
mv app.js src/app.js && git add src/app.js && git rm app.js
```

🤖 نکته ظریف: حتی اگر بدون `git mv` با `mv` معمولی جابه‌جا کنی و بعد `add` و `rm` کنی، گیت موقع کامیت خودش تشخیص می‌ده که `rename` بوده (`git status` می‌نویسه `renamed:`). پایاهش در فصل ۴ دیدی: هش از محتوا است، نه از اسم – پس فایل جابه‌جا شده، همون blob قبلیه. ولی `git mv` خواناتر و کم‌خطاتره.

حذف «فقط از مخزن» (فایل روی دیسک بمونه – مثل `.env` ای که track شده): `git rm --cached` – کاملش در فصل ۹.

۳.۳ Staging – پیشرفته: انتخاب تکه‌تکه‌ای

قابلیتی که خیلی‌ها نمی‌شناسن ولی در کار واقعی هر هفته لازم می‌شه: فایل‌ی داری که دو تغییر جداگانه توشه (مثلاً یک فیکس + یک ریفاکتور) و می‌خوای در دو کامیت جدا بیان:

```
git add -p app.js
```

گیت هر تغییر رو **hunk** (تکه) می‌بینه و می‌پرسه:

```
Stage this hunk [y,n,q,a,d,s,e,?]?
```

جواب	یعنی
y	این تکه بذار توی سبد
n	این تکه نه
s	تکه رو کوچیک‌تر تقسیم کن
q	کافیست، خارج شو
e	خودم دستی ویرایشش کنم

در VS Code هم همین کار گرافیکیه: پنل Source Control → روی فایل hover کن → «Stage Selected Ranges» یا با انتخاب خطوط در `diff`.

۳.۴ – کامیت: ثبت نهایی

```
git commit -m "پیام کوتاه یک خطی"

# (که تنظیم کردیم VS Code – ادیتور باز می‌شه) پیام بلند
git commit
```

یا میان‌بر: `git commit -a -m "..."` = همه فایل‌های tracked رو add و کامیت کن. ⚠ فقط برای تغییرات ریز OK است – `-a` فایل‌های جدید (untracked) رو نمی‌گیره و استفاده بی‌دقت ازش همون «کامیت‌های قاطی‌پاتی» رو می‌سازه که الان باهاشون مبارزه می‌کنیم!

اصلاح کامیت: `-- amend`

```
# پیام رو اشتباه نوشتی؟
git commit --amend -m "پیام درست"

# فایل رو جا انداختی؟
git add forgotten.js
git commit --amend --no-edit # به کامیت قبلی اضافه کن، پیام همون

# هم می‌شه عوض کرد (وقتی ایمیل شرکتی رو تنظیم نکرده بودی) author/تاریخ
git commit --amend --reset-author
```

⚠ قانون طلایی amend: فقط تا قبل از push! بعد از push یعنی بازنویسی تاریخچه‌ای که بقیه هم دارن (فصل ۸ کامل توضیح می‌ده چرا خطرناکه).

۳.۵ – هنر کامیت بامعنا 🏆

اینجا همون جاییه که «کامیت بldم» با «حرفه‌ای» فرق می‌کنه. تو تیم‌ها، تاریخچه کامیت‌ها کارت ویزیت توئه.

سه اصل

۱) اتمیک: هر کامیت = یک تغییر منطقی واحد. «فیکس باگ لاگین» و «آپدیت پکیج‌ها» و «پنل ادمین» = سه کامیت، نه یک کامیت ۳۰۰ خطی به اسم `update`.

۲) کامل: هر کامیت باید مستقل قابل فهم و *build*-پذیر باشه. نصف یک فیکس، خودش باگ جدیدیه.

۳) روشن: پیامش بگه چی و چرا – نه فقط چی.

✓ چیزی که تیم‌ها انتظار دارن	✗ در تاریخچه‌ات شاید داشته باشی
<code>fix(auth): prevent double submit on slow network</code>	<code>update</code>
<code>fix(cart): correct total when coupon removed</code>	<code>fix bug</code>
<code>refactor(api): extract validation to shared module</code>	<code>final!!</code>
<code>chore: bump next to 15.5, remove unused deps</code>	<code>asdasd</code>
<code>feat(payments): add Zarinpal webhook handler</code>	تغییرات

استاندارد صنعت: Conventional Commits

قالب رسمی که اکثر شرکت‌ها (و ابزارها) قبولش دارن:

`<type>(<scope>): <imperative توضیح کوتاه>`

کی استفاده می‌شه	type
فیچر جدید (نسخه !minor)	<code>feat</code>
رفع باگ	<code>fix</code>
تغییر کد بدون تغییر رفتار	<code>refactor</code>
مستندات فقط	<code>docs</code>
تست‌ها	<code>test</code>
ابزار، بیلد، دپندنسی	<code>chore</code>
بهبود پرفورمنس	<code>perf</code>
فرمت/فاصله — بدون تغییر منطق	<code>style</code>

`feat(checkout): add discount code support`

Coupons can now be applied **in** the cart step; totals recalculate on removal. Refunds keep the coupon **history for** auditing.

Closes [#142](#)

- خط اول: حداکثر ۵۰ کاراکتر، امری (مثل «add» نه «added»), بدون نقطه آخر
- بدنه (اختیاری): چرا این تغییر؟ چه راه‌حل جایگزینی رد شد؟

- footer: ارجاع به ایشو (Closes #142) – گیت‌هاب خودکار ایشو رو با merge می‌بنده

💡 چرا این استاندارد ارزش داره؟ (۱) changelog خودکار ساخته می‌شه، (۲) نسخه‌گذاری semantic (۳) git lg شما می‌شه یک خلاصه خوانا از پروژه. در ارزیابی‌های فنی و مصاحبه‌ها نیز همین تاریخچه تمیز، نشان‌دهنده سطح بالای نظم و بلوغ مهندسی تیم یا توسعه‌دهنده است.

۳.۶ – گردش کامل روزانه (چرخه‌ای که تا آخر دوره تکرار می‌شه)

```
git st          # چه خبر؟
#..            # کد می‌نویسی
git diff        # مرور کار خودم قبل از ثبت
git add -p     # انتخاب دقیق تکه‌ها
git commit -m "fix(auth): prevent double submit"
git lg -3      # مرور تاریخچه‌ای که ساختم
```

سه کامیت از یک روز کاری سالم:

```
a1b2c3d fix(auth): prevent double submit on slow network
e4f5g6h feat(cart): show item count badge in header
i7j8k9l chore: update eslint config, fix warnings
```

مقایسه کن با update update asd – ریویو کدوم تیم رو راحت‌تر قبوله؟ 😊

جمع‌بندی فصل ✓

- status/diff/log = چشم‌های تو؛ قبل و بعد از هر عمل صدا بزن
- git mv و git rm جابه‌جایی/حذف را ثبت می‌کنند؛ گیت rename را از هش محتوا خودش هم تشخیص می‌دهد
- diff (بی‌فلگ) = هنوز به سبد نرفته؛ -- staged = توی سبد؛ HEAD = جمع کل
- git add -p برای تقسیم تغییرات یک فایل بین چند کامیت
- --amend برای اصلاح آخرین کامیت – فقط قبل از push
- کامیت = اتمیک + کامل + پیام Conventional – این استاندارد رو از همین امروز در پروژه‌هات بزن
- footer ایشو را خودکار می‌بندد Closes #142

تمرین فصل ۳

1. در زمین تمرین: دو فایل بساز (`a.js` , `b.js`)، هر دو رو ویرایش کن، فقط `a.js` رو کامیت کن و بعد `b.js` .
خروجی `git lg --oneline` چی می‌شه؟
2. پیام آخرین کامیتت رو با `-- amend` اصلاح کن به فرمت Conventional.
3. یک فایل با سه تغییر جدا بساز (سه خط مختلف در جاهای مختلف) و با `git add -p` آنها را در دو کامیت جدا بذار.
4. یک فایل را با `git mv` به زیرپوشه ببر و کامیت کن — `git status` قبل از کامیت چه می‌گوید؟ بعد یک بار با `mv` معمولی تکرار کن: فرقی چی بود؟
5. آداب تمرینی بزرگ ★: از امروز، پروژه بعدیات را کاملاً با Conventional Commits بنویس. سه هفته بعد به `git lg` اش نگاه کن — خودت متوجه فرقی می‌شی.

▼ جواب تمرین ۱

```
git add a.js && git commit -m "feat: add module a"
git add b.js && git commit -m "feat: add module b"
git lg --oneline
# b2c3d4e (HEAD → main) feat: add module b
# a1b2c3d feat: add module a
```

هر کامیت یک snapshot مستقل است؛ می‌توانی با `git checkout a1b2c3d -- b.js` حال وجود نداشتن `b` را هم ببینی.

➔ فصل بعد: اشتباه کردی؟ نگران نباش — بازیابی‌ها بدون ترس: `revert`، `reset`، `restore` و `reflog`.

فصل ۴ – پشت صحنه: گیت واقعاً چطور کار می‌کند؟ (تور. git)

🎯 هدف: بعد از این فصل، وقتی به پوشه `.git` نگاه می‌کنی دیگه جعبه‌سیاه نمی‌بینی. همین فصل تفاوت «کسی که گیت رو حفظ کرده» با «کسی که گیت رو فهمیده» است – هم در حل مشکلات، هم در مصاحبه.

۴.۱ – چرا زیر پوست گیت بریم؟

سه دلیل کاملاً عملی: - وقتی چیزی گم می‌شه (`reflog` فصل ۶، `gc`، آبجکت‌های ناقص)، کسی که می‌دونه گیت چی رو کجا ذخیره می‌کنه، ده دقیقه‌ای حلش می‌کنه - پیام‌های ترسناک مثل `fatal: bad object` یا `index lock` دیگه وحشت‌آور نیستن - سوال مصاحبه محبوب: «گیت چطور فایل‌ها رو ذخیره می‌کنه؟» - جوابش کل این فصله

۴.۲ – تور پوشه `.git`

یادته با `git init` گفته شد «مخزن ساخته شد»؟ مخزن یعنی همین پوشه:

```
cd git-playground
ls -F .git/
# HEAD          ← فعلاً یک فایل متنی ساده! (پایین می‌بینیم)
# config        ← می‌زنی git config --local چیزهایی که با تنظیمات این مخزن
# hooks/        ← اسکریپت‌های خودکار - فصل ۱۴ بهش می‌پردازیم
# index         ← (بخش ۴.۵) Staging Area! فایل بایتری: همون
# objects/      ← دیتابیس اصلی گیت - همه چیز اینجاست
# refs/         ← (SHA! فقط) اشاره‌گرها: شاخه‌ها و تگ‌ها
# info/         description ...
```

دو تا از این‌ها رو همین حالا باز کن:

```
cat .git/HEAD
# ref: refs/heads/main          ← فقط یک آدرس اشاره‌گر است HEAD

cat .git/refs/heads/main
# a1b2c3d4e5f6...              ← داخلش SHA یک فایل متنی با یک main شاخه
```

🦉 لحظه‌ی «آهان»: شاخه‌ای که تا حالا ارزش حسابی می‌ترسیدی، فقط یک فایل ۴۱ بایتی متنی است که اسم یک کامیت توشه. ساختن شاخه = ساختن یک فایل کوچک. به همین دلیل شاخه زدن در گیت «ارزون»ه و تشویق می‌شی زیاد بزنی (فصل ۷).

۴.۳ – دیتابیس آبجکت‌ها: blob، tree، commit

گیت یک دیتابیس محتوا-محور (content-addressable) است: هر چیزی که ذخیره می‌شه، اول هش SHA-1 از محتواش گرفته می‌شه و همون هش، آدرسش می‌شه.

خودت امتحان کن – این دستور بدون هیچ مخزنی هم کار می‌کنه:

```
echo "hello git" | git hash-object --stdin
# 8ab686eafeb1f44702738c8b0f24f2567c36da6d ← همین محتوا، همیشه همین هش
```

چهار نوع آبجکت داریم:

آبجکت	چی ذخیره می‌کنه	مثال...
blob	محتوای یک فایل (بدون نام فایل!)	محتویات جعبه
tree	یک پوشه: اسم فایل‌ها + اشاره به blob و tree های زیرین	برچسب روی جعبه‌ها
commit	یک snapshot: اشاره به یک tree + والد (ها) + پیام + author + تاریخ	برگه‌ی ثبت
tag	فقط برای تگ annotated: اشاره به کامیت + پیام + امضا (فصل ۹)	مهر روی برگه

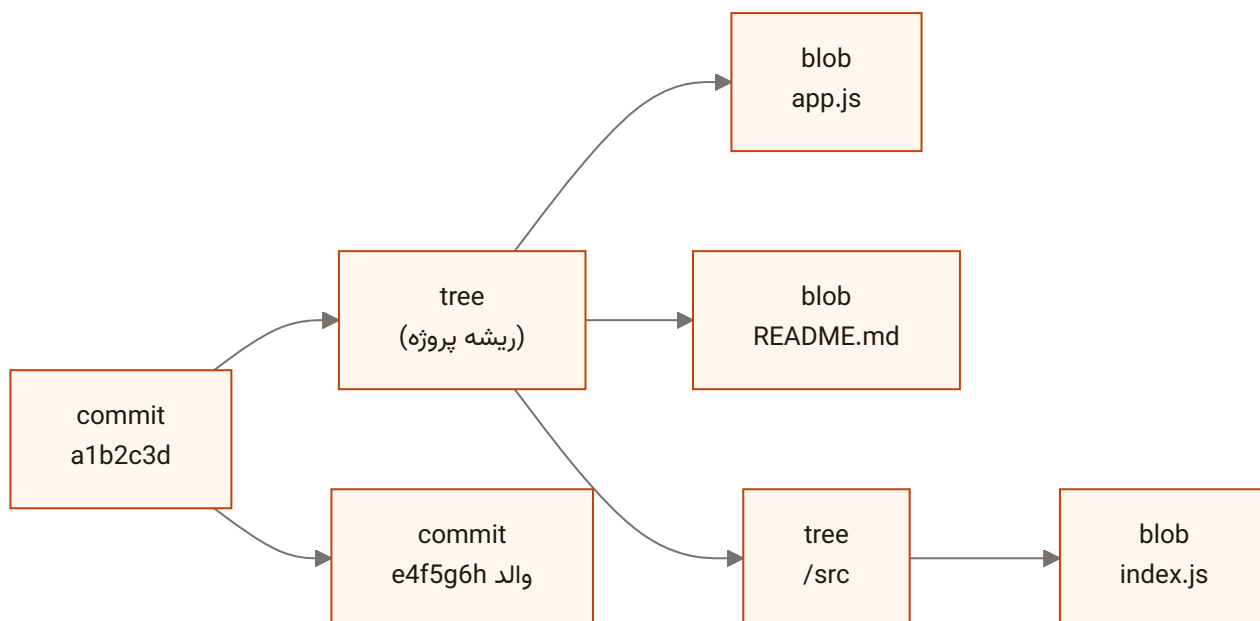
ابزارهای X-Ray برای دیدن این‌ها:

```
git cat-file -t a1b2c3d # نوع آبجکت: commit / blob / tree / tag
git cat-file -p a1b2c3d # محتوای خوانا (p = pretty)
git ls-tree HEAD # هابی blob درخت ریشه: چه فایل‌هایی، با چه
git ls-tree HEAD src/ # داخل یک پوشه
git rev-parse main # کامل یک شاخه SHA
```

خروجی `git cat-file -p HEAD` یک کامیت واقعی شبیه اینه:

```
tree 4b825dc642cb6eb9a060e54bf8d69288fbee4904
parent e4f5g6h... ← کامیت قبلی (کامیت اول والد ندارد)
author Sarah Dev <sarah@example.com> 1726300000 +0330
committer Sarah Dev <sarah@example.com> 1726300000 +0330

feat: add search endpoint
```



💡 نکته کلیدی: چرا گیت rename را خودش تشخیص می‌دهد – داخل blob نه اسم فایل هست نه مسیرش؛ اسم‌ها در tree ذخیره می‌شن. پس وقتی فایلی جابه‌جا می‌شه، محتوای blob رو می‌شناسد و می‌گه «این همون فایل قبلیه، فقط عوض شده جاش» (تمرین فصل ۳).

۴.۴ – چرا snapshot ارزون است؟ (dedup)

فصل ۱ گفتیم هر کامیت = snapshot کامل، نه diff. این با «دیتابیس محتوا-محور» جور درمیاد:

- دو فایل با محتوای یکسان (حتی با اسم‌های متفاوت) → یک blob مشترک، چون هاش‌شون یکیه
- در کامیت جدید، فایل‌هایی که تغییر نکردن → کامیت جدید فقط به همان blob های قبلی اشاره می‌کنه

پس پروژه‌ی ۵۰ مگابایتی با هزار کامیت، ۵۰ گیگ نمی‌شه – فقط محتوای عوض‌شده دوباره ذخیره می‌شه. snapshot بودن، هم به سرعت switch و reset کمک می‌کنه (کپی دیتابیس لازم نیست، فقط اشاره‌گرها جابه‌جا می‌شن) هم به سبک ماندن مخزن روی دیسک.

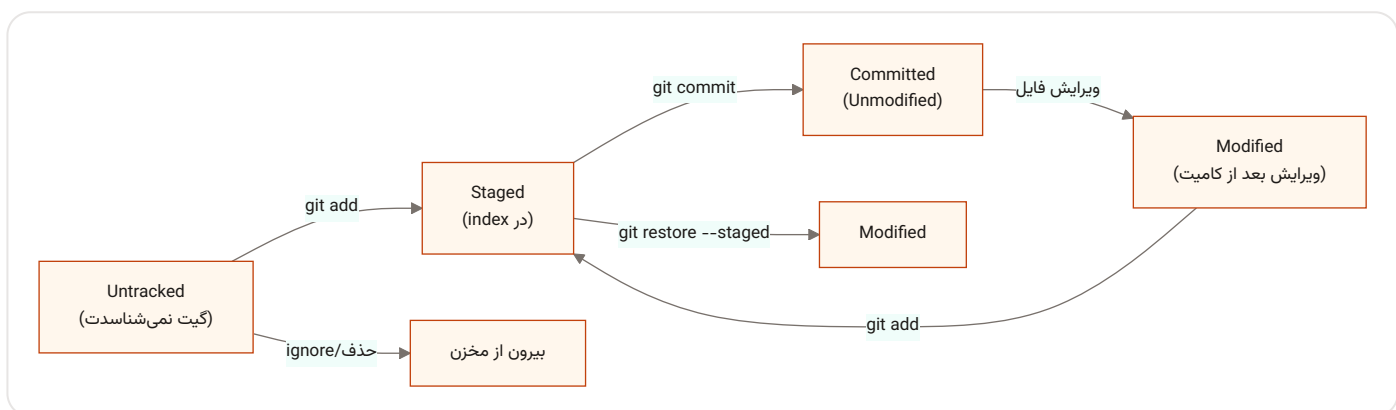
۴.۵ – index: Staging Area از نگاه گیت

اونا فایل باینری `.git/index` – چیزی که اسمش staging area است – یک «قرار snapshot بعدی» است: لیستِ دقیقِ این‌که در کامیت بعدی چه محتوایی با چه اسمی باشد.

سه دستور پایه، سه حرکت زیر پوست:

دستور	در چشم گیت
<code>git add app.js</code>	محتوا → یک blob جدید در <code>objects/</code> + ثبتش در index
<code>git commit</code>	از index یک <code>tree</code> بساز + یک آبجکت <code>commit</code> به آن + جلو بردن اشاره گر شاخه (یعنی <code>refs/heads/main = SHA</code> جدید)
<code>git restore --staged app.js</code>	فقط ورودی <code>app.js</code> را از index حذف کن (blob در <code>objects</code> می ماند – بی ضرر)

و چرخه حالت های فایل – همون جدولی که بهش ارجاع زیاد می دیم:



🎯 وقتی `status` می گیری، گیت در واقع دو مقایسه می کنه: **index** در برابر آخرین کامیت (فایل های Staged) و **working directory** در برابر **index** (فایل های Modified/Untracked). حالا دیگه می دونی چرا «سه ناحیه» سه تاست.

۴.۶ – `git gc`، `packfile` و هوای تازه دادن به مخزن

آبجکت های تکی (loose objects) هر کدوم یک فایل جدا در `objects/` هستند. گیت وقتی تعدادشون زیاد می شه (یا موقع `push`)، اون ها رو **فشرده و بسته بندی** می کنه: `objects/pack/*.pack` – به علاوه `diff` های محاسبه شده بین blob های مشابه (delta compression). برای همین مخزن های بزرگ خیلی کوچک تر از جمع فایل هاشونند.

نگهبان این چرخه:

```

git count-objects -vH    # حجم loose، چند pack، چند آبجکت
git gc                   # ★ garbage collection: پاکسازی +
git gc --prune=now        # همین حالا آبجکت های بی مرجع را هم پاک کن

```

- گیت خودش گاه گاهی (بعد از عملیات های حجیم) `gc` خودکار می زنه – معمولاً لازم نیست دستی بزنی
- `gc` آبجکت های «بی مرجع» رو پاک می کنه؛ ولی `reflog` ها (فصل ۶) یک مهلت نگهداری دارن (پیش فرض ~۳۰/۹۰ روز) – یعنی «کامیت گم شده» معمولاً هنوز نجات پذیره، وگرنه عمر `reflog` هم محدوده

- `git fsck --lost-found` جاروی آخری برای پیدا کردن آبجکت‌های یتیم (اگر reflog هم پاک شده بود)

🔔 **قانون خانه‌داری: داخل `.git` هیچ‌وقت دستی چیزی پاک یا ویرایش نکن** – کل استثنا: پوشه `hooks/` (فصل ۱۴) و دستورات رسمی مثل `gc`. تمیزکاری با گیت انجام می‌شه، نه با Explorer.

✓ جمع‌بندی فصل

- مخزن = پوشه `.git`: دیتابیس `objects/` + اشاره‌گرهای `refs/` + فایل `index + HEAD`
- شاخه = یک فایل متنی حاوی SHA؛ ساخت شاخه ارزون است چون فقط اشاره‌گر ساخته می‌شه
- ۴ نوع آبجکت: blob (محتوا)، tree (پوشه)، commit (snapshot+والد)، tag (مهر annotated)
- هاش از محتوا: محتوای یکسان → blob مشترک (dedup) → snapshot ها ارزون‌اند
- `add` = ساخت blob و ثبت در `index`؛ `commit` = ساخت tree و commit و جلو بردن شاخه
- `git gc` بسته‌بندی و پاکسازی می‌کند؛ داخل `.git` دست‌کاری ممنوع (به‌جز hooks)

📝 تمرین فصل ۴ (همه در زمین تمرین – ریسک صفر)

1. در `.git` تور بزن: `cat .git/HEAD` و `cat .git/refs/heads/main` – بعد `git rev-parse main` بزن و مقایسه کن. برابره؟
2. دو فایل با محتوای یکسان ولی اسم متفاوت بساز، هر دو را `add` کن، بعد `git ls-files -s` بزن – چرا فقط یک SHA می‌بینی؟
3. `git cat-file -t <sha>` و `git cat-file -p <sha>` را روی کامیت آخر، روی tree آن، و روی یکی از blob هایش اجرا کن (زنجیره blob → tree → commit را دستی دنبال کن).
4. `git count-objects -vH` بزن، چند کامیت بزن، دوباره بزن؛ بعد `git gc` و دوباره مقایسه کن – حجم چطور شد؟
5. (چالش) بدون `git switch` و بدون checkout: با `<یکی-کامیت-قدیمی>` `git cat-file -p <sha>` محتوای یک فایل در آن نسخه را ببین.

```
# تمرین ۲:
git ls-files -s
# 100644 8ab686ea 0 app.js
# 100644 8ab686ea 0 copy-of-app.js    ← همان SHA! یک blob، دو اسم (index هر دو در)
# نکته می‌دارد blob چون هش از «محتوا» است و محتواها یکی‌اند، گیت فقط یک.

# تمرین ۵:
git log --oneline          # کامیت قدیمی موردنظر SHA
git cat-file -p <sha>      # را کپی کن tree خط
git cat-file -p <tree-sha>  # را پیدا کن (blob) فایل SHA
git cat-file -p <blob-sha>  # !محتوای فایل در آن نسخه، بدون جابه‌جایی شاخه ←
```

➔ **فصل بعد:** حالا که می‌دونی تاریخچه یعنی زنجیره آبجکت‌ها، یاد می‌گیریم مثل حرفه‌ای‌ها ازش جست‌وجو بزنیم – `git log` نسخه کامل.

فصل ۵ – git log: تاریخچه خوانی مثل حرفه‌ای‌ها

🎯 هدف: `git log` رو از «لیست کامیت‌های اسکرول‌شونده» به یک موتور جست‌وجوی قدرتمند تبدیل می‌کنیم: فیلتر بر نویسنده/زمان/فایل، شکار یک رشته در کل تاریخچه، و خروجی قالب‌بندی‌شده مال خودت.

۵.۱ – چرا log از چیزی که فکر می‌کنی مهم‌تره

سناریوهای هر هفته‌ی کار واقعی: - قبل از `pull` یا `rebase` می‌خوای بدونی اون طرف آب چه خبره (فصل ۱۰) - «این فایل رو کی و برای چی تغییر داد؟» - قبل از هر `refactor` - می‌خوای متن PR خوب بنویسی (فصل ۱۱) - باید خودت لیست تغییرات رو ببینی - و در مصاحبه: تسلط روی `log` و فیلترهایش نشونه‌ی «کسیه که واقعاً با گیت زندگی می‌کنه»

۵.۲ – نمایش‌ها: از خروجی خام تا نقشه شبکه

```
git log                                # خارج شو q (صفحه‌ای - با) خروجی کامل
git log --oneline -10                 # کامیت آخر، فشرده ۱۰
git log --stat                        # با آمار فایل‌های تغییر کرده
git show HEAD                        # یک کامیت خاص + diff + جزئیات

# ★ دستور روزمره حرفه‌ای‌ها - نقشه شاخه‌ها
git log --graph --oneline --all --decorate
```

خروجی گزینه آخر چیزی شبیه اینه:

```
* a1b2c3d (HEAD → feature/login, origin/feature/login) feat: remember me
* e4f5g6h feat: login form validation
| * i7j8k9l (origin/main, main) fix: navbar overlap      ← جلوتره main یعنی!
|/
* m0n1o2p chore: bump deps                             ← نقطه مشترک
```

یک نگاه به این خروجی = می‌دونی شاخه‌ات از کجا جدا شده، main چقدر جلوتره و چه چیزی push شده. عادت کن قبل از هر `merge/rebase` اول همین رو بزنی.

۵.۳ – فیلترها: فقط چیزی که می‌خوای

```
git log --author="Sarah"             # فقط کامیت‌های یک نفر
git log --grep="login"                # جست‌وجو در پیام کامیت‌ها
git log --since="2 weeks ago"         # (هم داره until -) بازه زمانی
git log -- src/app.js                 # فقط کامیت‌هایی که این فایل/پوشه را لمس کردند
git log --merges                      # merge commit فقط/بدون --no-merges یا
git log -5 --skip=5                   # صفحه‌بندی دستی (۵ تای دوم)
```

★ مهم‌ترین‌ها: رنج شاخه‌ها – قبل از ساخت PR، کامیت‌های اختصاصی شاخه فیچر نسبت به شاخه اصلی را مرور کنید:

```
git log main..feature/login # نیست main هست، در feature کامیت‌هایی که در
git log feature/login..main # جلوتر است (بدهی‌ای که باید بگیری) main کامیت‌هایی که
git log main...feature/login # هر دو طرف، با علائم > و < مشخص می‌شود
```

💡 قانون ساده حفظی: **A..B** یعنی «توی B هست، توی A نیست». خودت را جای **A = main** و **B = شاخه‌ام**

بذار تا اشتباه نشه. این همون خروجیه که صفحه Compare گیت‌هاب (فصل ۱۱) بهت نشون می‌ده – اینجا کنترلش دست خودته.

۵.۴ – شکار تغییرات: pickaxe و -p

سوال کلاسیک: «این **apiKey** از کجا وارد کد شد؟» فیلتر پیام (**--grep**) فقط در پیام می‌گرده؛ **pickaxe** در محتوای تغییرات می‌گرده:

```
git log -S "apiKey" --oneline # کامیت‌هایی که تعداد این رشته را کم/زیاد کردند ★
git log -G "getPrice\s*\(" --oneline # روی خط‌های تغییر کرده regex با
git log -p -- src/config.js # کامل هر کامیت diff خروجی با
git log -p -1 # diff آخرین کامیت با
git log -w -p -- src/app.js # نادیده گرفته شود (خروجی تمیزتر) whitespace تغییرات
```

ابزار	کی استفاده کن
(pickaxe) -S "str"	وقتی می‌دونی چه رشته‌ای عوض شده – «این ثابت از کجا اومد؟»
-G "regex"	الگوی پیچیده‌تر روی خود خطوط
-p	می‌خواهی خودِ diff رو ببینی، نه فقط فهرست
git bisect (فصل ۹)	وقتی فقط علامت باگ رو می‌دونی و عاملش رو نه – جست‌وجوی دودویی

۵.۵ – قالب‌بندی: خروجی مال خودت

```
git log --pretty=format:"%h %an (%ad) %s" --date=short
# a1b2c3d Sarah (2026-09-12) feat: remember me
```

جایگاه‌های پرکاربرد:

جایگاه	معنی	جایگاه	معنی
%h / %H	هش کوتاه / کامل	%an / %ae	نام / ایمیل نویسنده
%ad / %ar	تاریخ مطلق / نسبی (۲ روز پیش)	%s / %b	عنوان / بدنه پیام
%d / %D	رفرنس‌ها (HEAD -> main) داخل / خارج خط	%C(...)	رنگ - تا %C(reset)

بهترین قالب، همونیه که همیشه لازم داری - یک بار به صورت alias ذخیرهش کن (سبک alias های فصل ۲):

```
git config --global alias.lg 'log --graph --oneline --decorate --all'
# از این به بعد فقط: git lg

git config --global alias.who 'log --pretty=format:"%h %an %ar %s" -10'
# و: git who
```

۵.۶ - blame و shortlog: «کی نوشته؟»

```
git shortlog -sn                # آمار: چند کامیت به ازای هر نفر (رتبه‌بندی مخزن!)
git shortlog -sn --since="1 month"

git blame -- src/app.js         # هر خط: کدوم کامیت، کی، کی‌دور
git blame -L 10,20 -- src/app.js # فقط خط‌های ۱۰ تا ۲۰
git blame -w -- src/app.js      # در نظر بگیر formatting-کامیت‌های فقط
```

📌 ادب حرفه‌ای blame: این ابزار برای «مقصر تراشیدن» نیست - برای «فهمیدن زمینه» است. خط عجیبی دیدی؟ بین کدوم کامیت، برای کدوم PR/موضوع نوشته شده (کد کامیت رو در `git show` یا URL گیت‌هاب باز کن) و بعد با صاحبش حرف بزن. GitLens در VS Code همین اطلاعات رو قشنگ‌تر و زنده نشون می‌ده (فصل ۱۵).

✅ جمع‌بندی فصل

- روزمره: `git log --graph --oneline --all --decorate` - نقشه کامل شاخه‌ها در یک نگاه
- فیلترها: `<path>` ، `-- since` ، `-- grep` ، `-- author` ، و رنج‌ها `main..feature` (قبل از هر PR)
- (pickaxe) `-S` یک رشته را در کل تغییرات تاریخچه شکار می‌کند؛ `-p` خود diff را می‌دهد
- (`git lg`) `alias` + `--pretty=format:` = خروجی تمیز مادام‌العمر با یک بار تنظیم
- `shortlog -sn` آمار تیم؛ `blame` برای فهم زمینه، نه مقصرتراشی

تمرین فصل ۵ (همه در زمین تمرین)

1. زمین تمرین را با `git lg` ببین (`alias` را از بخش ۵.۵ بساز). چند کامیت بزن، یک شاخه بزن و `merge` کن — نقشه `graph` چه شکلی شد؟
2. با `-s` پیدا کن کامیت معرفی یک رشته‌ی خاص در زمین تمرین (مثلاً یک تابع که قبلاً نوشتی). بعد با `git show` جزئیاتش را ببین.
3. `main..feature` و `feature..main` را برای یک شاخه قدیمی‌ات اجرا کن و تفاوت دو خروجی را توضیح بده.
4. (چالش) با `shortlog -sn` روی یک مخزن اوپن‌سورس واقعی (مثلاً یکی از پروژه‌هایی که در فصل ۱۰ فورک می‌کنی) ببین ۵ نفر اول کی‌اند؛ بعد `since="1 year"` بزن — چه تغییری دیدی و چرا؟

▼ جواب تمرین ۳

```
git log main..feature/login      # کامیت‌هایی که در این شاخه اضافه شده و هنوز در شاخه مبنا نیست
git log feature/login..main      # اضافه کردند main بدهی: کامیت‌هایی که بقیه به
                                # (rebase/merge) و شاخه فیچر ندارد → قبل از ادغام باید دریافت و همگام شوند
```

اولی «چیزی که می‌دم» است، دومی «چیزی که باید بگیریم». اگر دومی خالی بود، PR ات بدون تعارض `merge` می‌شه.

→ **فصل بعد:** با این دید تازه، سراغ برگرداندن‌ها می‌رویم — `revert`، `reset`، `restore` و بیمه‌ی زندگی: `reflog`.

فصل ۶ – برگرداندن‌ها: بدون ترس! (reset، revert، restore، reflog)

🎯 هدف: بزرگ‌ترین منبع استرس گیت، «ترس از خراب کردن» است. این فصل جدول تصمیم کامل undo رو بهت می‌ده و reflog رو یادت می‌ده – ابزاری که عملاً یعنی «هیچ کامیت‌ای گم نمی‌شه».

۶.۱ – جدول تصمیم: چه چیزی رو می‌خوام برگردونم؟

قبل از هر دستوری، خودت رو از این سوال رد کن:

وضعیت	راه‌حل	ایمنی
ویرایش فایل رو پشیمونم (هنوز add نشده)	<code>git restore file</code>	⚠️ تغییر برای همیشه می‌ره
فایل رو اشتباه add کردم	<code>git restore --staged file</code>	✅ بی‌خطر – فقط از سبد خارج می‌شه
آخرین کامیت پیامش/محتواش ایراد داره	<code>git commit --amend</code>	✅ فقط قبل از push
کامیت آخر رو کلاً نمی‌خوام، کد کارم بمونه	<code>git reset --soft HEAD~1</code>	✅ تغییرات staged می‌مونه
کامیت push شده اشتباهه (تاریخچه مشترک!)	<code>git revert</code>	✅ کامیت معکوس – امن برای تیم
یه کامیت قدیمی رو باید بی‌خیال شم	<code>git revert <sha></code>	✅
«بروم به چیز رو امتحان کنم»	<code>git switch -c experiment</code>	✅ شاخه! بهترین undo دنیا

۶.۲ – restore: برگرداندن فایل‌ها

```
# این فایل رو برگردون به آخرین کامیت working directory ویرایش‌های
git restore app.js

#!کدت دست نمی‌خوره (فقط unstage – کامیت خارج کن)
git restore --staged app.js

# هر دو با هم: برگرد به وضعیت آخرین کامیت
git restore --staged --worktree app.js
```

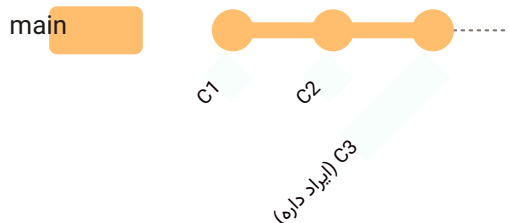
🔥 تنها دستور «خطرناک» این خانواده، `restore` روی `worktree` است: ویرایش‌های ذخیره‌نشده واقعاً می‌پرن. اگر مطمئن نیستی، اول شاخه بزن (بخش ۶.۵) یا `git stash` کن (فصل ۹).

۶.۳ – `reset`: جابجایی شاخه (سه حالت)

`git reset` شاخه فعلی رو به یک کامیت دیگه می‌بره. سوال اینه: کارهای میانی چی بشن؟ سه حالت:

```
git reset --soft HEAD~1  # می‌مونه staged: شاخه می‌ره عقب؛ تغییرات
git reset HEAD~1         # working directory: تغییرات: توی (پیش‌فرض - mixed)
git reset --hard HEAD~1  # همه چیز پاک! به کامیت قبل برگرده، هیچی نمونه ⚠️
```

تصویری برای `HEAD~1` (کامیت قبل از HEAD):



بعد از `git reset --soft HEAD~1`:



Working	Staging	Repository	حالت
می‌مونه ✓	می‌مونه ✓	می‌ره عقب	<code>--soft</code>
می‌مونه ✓	خالی می‌شه	می‌ره عقب	<code>--mixed</code> (پیش‌فرض)
پاک می‌شه ⚠️	خالی	می‌ره عقب	<code>--hard</code>

سناریوی واقعی: «سه کامیت اول صبح رو قاطی زدم، می‌خوام همه رو یک کامیت تمیز بزنم»:

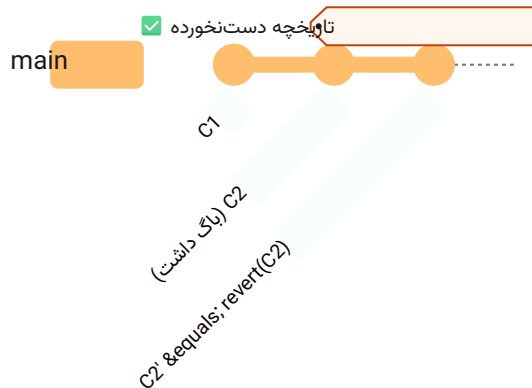
```
git reset --soft HEAD~3  # باقی ماند Staged هر سه کامیت لغو شد، اما تغییرات کد در حالت
git commit -m "feat(auth): complete login flow with tests"
```

⚠️ قانون طلایی **reset**: هرگز روی شاخه‌ای که **push** شده و بقیه روش کار می‌کنن، **reset** نزن — تاریخچه رو بازنویسی می‌کنه و **push** بعدی باید **force** بشه (فصل ۸: قانون طلایی **rebase** — همین قانونه).

۶.۴ — **revert**: «برگردن امن» برای تاریخچه مشترک ★

revert تاریخچه رو نمی‌شکنه: یک کامیت جدید معکوس می‌سازه که اثر کامیت هدف رو خنثی می‌کنه:

```
git revert a1b2c3d      # تأیید کن "Revert '...' ادیتور باز می‌شه: پیام
git revert --no-edit a1b2c3d
```



فرق با **reset** یک خطی است: **reset** تاریخچه رو بازنویسی می‌کنه (لوکال/شخصی)، **revert** بهش اضافه می‌کنه (عمومی/مشترک). در شرکت، روی شاخه‌های مشترک فقط **revert**!

مشخص کن کدوم والد نگه داشته شه **m** با — (شده **merge** اشتباهی **PR** مثلاً) یک **merge commit** **revert**:

```
git revert -m 1 <merge-sha>
```

۶.۵ — **reflog**: بیمه زندگی ★★

این بخش رو خوب بخون، چون همین جاست که «ترس گیت» می‌میره.

گیت یک ژورنال حرکات **HEAD** نگه می‌داره — حتی **reset** های **hard** و **rebase** ها رو:

```
git reflog
# a1b2c3d (HEAD → main) HEAD@{0}: reset: moving to HEAD~1
# e4f5g6h HEAD@{1}: commit: feat: add search      ← «کامیت» گم شده!
# i7j8k9l HEAD@{2}: checkout: moving from main
```

کامیت‌هایت تا ~۹۰ روز در **.git** زنده‌ان حتی اگه هیچ شاخه‌ای بهشون اشاره نکنه. برگردوندنش:

```
git reset --hard e4f5g6h      # یا: git reset --hard HEAD@{1}
```

همین. «کامیت رو با `reset --hard` پاک کردم!» – نه، فقط اشاره‌گر بردی؛ `reflog` برش می‌گردونه.

💡 پیش‌نمایش فصل ۱۳: وقتی نالا و استرس‌زاست (مثلاً وسط جلسه با لید)، اول `git reflog` بزن. تقریباً هر فاجعه‌ای با این یکی دستور خنثی می‌شه. نفس عمیق، بعد `reflog`.

۶.۶ – پاکسازی فایل‌های بی‌ردیب

```
git clean -n    # Dry-run: چه فایل‌هایی untracked شدند؟
git clean -fd    # untracked حذف واقعی فایل‌ها و پوشه‌های
```

⚠️ `clean` برخلاف بقیه گیت، برگشت‌ناپذیره (untracked ها هیچ snapshot ای ندارند). همیشه اول `-n`.

✅ جمع‌بندی فصل

- جدول تصمیم رو حفظ کن: پشیمونی ویرایش `→ restore`؛ اشتباه `--staged` `→ add`؛ اصلاح `→ amend`؛ کامیت لوکال اضافه `→ reset --soft`؛ کامیت مشترک `→ revert`
- `reset` سه حالت: `soft` (همه‌چیز می‌مونه)، `hard`، `mixed` (unstage) (پاک – خطرناک)
- `revert` = کامیت معکوس؛ تنها راه امن برگردوندن روی شاخه‌های `push` شده
- `reflog` = بیمه زندگی – هیچ کامیت `~۹۰` روز از بین نمی‌ره
- `git clean` تنها برگشت‌ناپذیره – همیشه `-n` اول

📝 تمرین فصل ۶ (در زمین تمرین – جرئت داری، خراب می‌کنی برمی‌گردونی!)

1. سه کامیت بزن. آخرینش رو با `reset --soft` بردار، بعد با پیام Conventional دوباره بزن.
2. یک کامیت رو `revert` کن و با `git lg` ببین چه اتفاقی افتاد (تاریخچه چند خط شد؟).
3. یک کامیت بزن، `reset --hard HEAD~1` بزن (کامیت «پاک» شد!)، حالا با `git reflog` پیداش کن و برگردونش. این تمرین رو تا جایی تکرار کن که «ترس»ت بره 😊
4. سوال: همکاری روی `main` مشترک `push` کرده و فهمیدن کامیت وسطی باگ داره. `reset` یا `revert`؟ چرا؟

▼ جواب تمرین ۳

```
echo x > f.txt && git add f.txt && git commit -m "test: will be lost"
git reset --hard HEAD~1
git lg --oneline -2          # کامیت نیست!
git reflog -5               # کامیت را اینجا پیدا کن SHA
git reset --hard <SHA>
git lg --oneline -2          # برگشت ✓
```

تمرین ۴: **revert** – چون کامیت push شده و تاریخچه مشترک؛ reset نیازمند force push است و کاری بقیه رو خراب می‌کند.

→ **فصل بعد:** قلب همکاری: شاخه‌ها، merge و – بدون ترس – حل تعارض.

فصل ۷ – شاخه‌ها، Merge و حل تعارض (بدون استرس!)

🎯 هدف: شاخه = کارت اعتباری رایگان. یاد می‌گیری چطور شاخه بزنی، merge کنی، و مهم‌تر از همه: merge conflict رو قدم‌به‌قدم و آروم حل کنی – همون چیزی که خیلی‌ها رو می‌ترسونه ولی در عمل یک کار مکانیکیه.

۷.۱ – چرا شاخه؟

سناریوی شرکت: روی `main` نسخه پایدار هست. تو باید فیکس کنی، همکاری فیچر می‌سازه، و ناگهان باگ حیاتی production گزارش می‌شه. بدون شاخه؟ فاجعه. با شاخه؟ هر کسی توی دنیای خودش:

- عایق‌سازی: کار نیمه‌تمامت به `main` لطمه نمی‌زنه

- موازی‌کاری: چند کار همزمان، چند نفر

- بازبینی: کار هر فیچر جدا قابل ریویو (PR – فصل ۱۱)

- بازگشت: فیچر شکست خورد؟ شاخه‌ش رو پاک کن، `main` سالمه

قاعده شرکت‌ها: هیچ‌وقت مستقیم روی `main` کامیت نزن (`main` معمولاً `protected` هم هست). همیشه `feature/*` یا `fix/*`.

۷.۲ – دستورات مدرن: `switch` و `branch`

دستورات مدرن گیت (از ۲.۲۳ به بعد) کار `checkout` رو به دو دستور روشن تقسیم کرده – از همینا استفاده کن:

```
git branch          # لیست شاخه‌ها (* = الان کجام)
git switch -c feature/search  # ساخت + پرش به شاخه جدید (-c = create)
git switch main      # برگشت به main
git switch -         # برگشت به شاخه قبلی (مثل cd -)
git branch -d feature/search  # شده merge حذف شاخه
git branch -D feature/search  # (!نجات پیدا می‌کنن reflog نشده می‌پرن - merge کامیت‌های) حذف اجباری
```

💡 نام‌گذاری رایج در شرکت‌ها: `feature/checkout-discount` ، `fix/login-404` ، `hotfix/urgent-payment` ،

`chore/deps-update` . از نام‌های نامشخص و شخصی مانند (`temp-work` یا `test-branch`) پرهیز کنید – محتوای کار رو اسم کن.

۷.۳ – Merge: دو خط تاریخچه یکی می‌شن

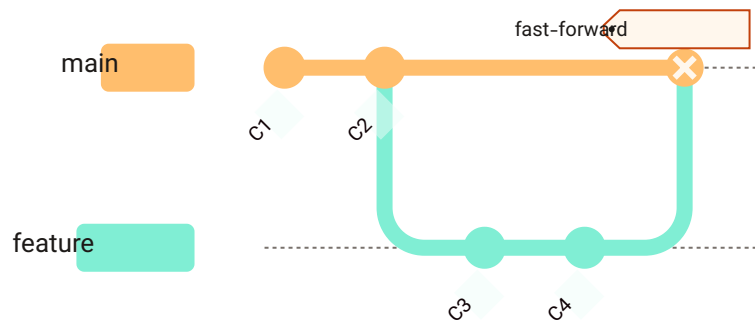
سناریوی زمین تمرین:

```
git switch -c feature/greeting
echo "console.log('hello')" > greet.js
git add . && git commit -m "feat: add greeting module"
git switch main
echo "v2" > version.txt
git add . && git commit -m "chore: bump version"
git merge feature/greeting
```

دو حالت merge وجود داره — دیدنش کلید فهم همه چیز است:

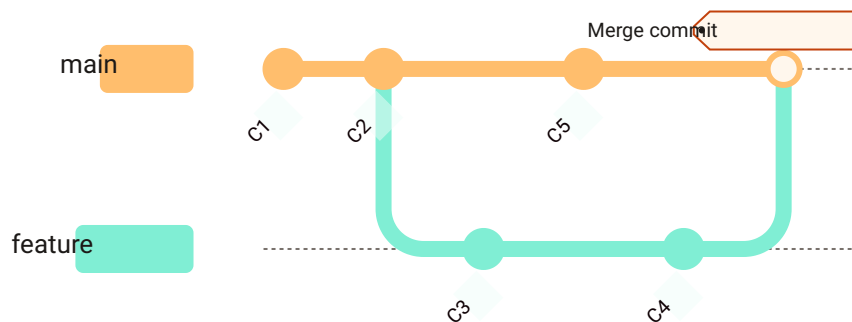
حالت ۱: Fast-forward

اگه از انشعاب main ثابت بوده (هیچ کامیت جدیدی نداشته)، گیت فقط اشاره گر رو جلو می‌بره:



حالت ۲: Merge commit (سه طرفه)

اگه هر دو طرف کامیت جدید داشتن، گیت یک کامیت merge می‌سازه که دو والد داره:



💡 در گیت‌هاب (PR) همیشه merge commit ساخته می‌شه (مگر strategy دیگه — فصل ۱۱). پس تصویر دوم، ظاهر استاندارد تاریخچه شرکت‌هاست.

۷.۴ Merge Conflict – رودررویی بی خطر 😊

تعارض وقتی که دو نفر یک خط یکسان رو متفاوت تغییر دادن – گیت نمی‌دونه کدوم حق داره و از تو می‌پرسه. این یک ارور نیست؛ یک سوال مودبانه‌ست!

بسازیمش (زمین تمرین):

```
git switch -c feature/tagline
echo "بهترین فروشگاه" > tagline.txt
git add . && git commit -m "feat: tagline from feature"

git switch main
echo "فروشگاه شماره یک" > tagline.txt
git add . && git commit -m "feat: tagline on main"

git merge feature/tagline
# CONFLICT (content): Merge conflict in tagline.txt
# Automatic merge failed; fix conflicts and then commit the result.
```

حالا `git status` بگو: `both modified: tagline.txt` . فایل رو باز کن:

```
<<<<<< HEAD
فروشگاه شماره یک
=====
بهترین فروشگاه
>>>>>> feature/tagline
```

حل تعارض قدم به قدم (آیین‌نامه رسمی آرومی):

۱. نفس بکش – هیچی از دست نرفته. `merge` ناتمام رو با `git merge --abort` هر لحظه می‌تونی لغو کنی و به وضعیت قبل برگردی.

۲. فایل‌های متعارض رو باز کن (`git status` لیستشون رو می‌ده). سه بخش رو بخون: `<<<<<<` تا `=====` = نسخه فعلی (HEAD)؛ `=====` تا `>>>>>>` = نسخه شاخه دیگر.

۳. تصمیم بگیر: یکی رو نگه دار، یا هر دو رو ادغام کن، یا با همکاری (که اون کد رو نوشته!) هماهنگ شو – این مهم‌ترین قدمه: تعارض یعنی دو تصمیم محصولی به هم خوردن؛ حلش کار فنی+انسانیه.

۴. نشان‌ها رو پاک کن و فایل رو به شکل نهایی دربیار:

فروشگاه شماره یک – بهترین انتخاب شما

۵. علامت‌گذاری حل:

```
git add tagline.txt
git status      # ببین همه متعارض‌ها حل شدن
git commit      # آماده داره – تأیید کن merge گیت پیام
```

۶. قبل از `commit`، تست کن که کد واقعاً کار می‌کنه (build/run) – تعارض حل‌شده ظاهری، باگ پنهان می‌سازه.

VS Code بالای بخش متعارض دکمه‌های **Accept Current / Accept Incoming / Compare** می‌ذاره — ولی توصیه: اول چند بار دستی نشان‌ها رو بخون و پاک کن تا «بدنی» درکش کنی؛ بعد از ابزار استفاده کن.

```
# دیدن لیست دقیق فایل‌های متعارض:
git diff --name-only --diff-filter=U
# merge عقب‌نشینی کامل از:
git merge --abort
```

🔔 **ضد استرس:** تعارض داده‌ها رو پاک نمی‌کنه — هر دو نسخه توی فایل و تاریخچه کامله. بدترین اتفاق ممکن: فایل رو به هم بریزی و **commit** اشتباه بزنی → هنوز با **git reflog** و **git reset** برمی‌گردی (فصل ۱۶).

۷.۵ — حذف و مرتب‌کاری بعد از merge

```
git branch -d feature/tagline  # شده دیگر لازم نیست merge شاخه
git lg                        # با گراف، شکل نهایی رو ببین
```

جمع‌بندی فصل ✓

- همیشه روی **main** مستقیم کار نکن؛ **feature/*** بزن — نام‌گذاری معنادار
- دستورات مدرن: **switch -c** (ساخت+پرش)، **switch -** (برگشت)، **branch -d** (حذف امن)
- **merge** دو صورت داره: **fast-forward** (خط مستقیم) و **merge commit** (دو والد)
- **تعارض = سوال، نه ارور.** آیین: بخون → تصمیم بگیر (با همکاری!) → نشان‌ها رو پاک کن → **add** → **commit** → تست
- دکمه لغو همیشه‌گیه — هر زمان می‌تونی عقب‌نشینی کنی **git merge --abort**

تمرین فصل ۷ 📝

1. زمین تمرین: شاخه **feature/colors** بزن، فایل **theme.css** بساز و کامیت کن؛ برگرد **main**، همون فایل رو با محتوای متفاوت بساز و کامیت کن؛ **merge** کن و تعارض رو دستی حل کن (بدون ابزار!). بعد با **git lg** شکل **merge commit** رو ببین.
2. **git branch -d** روی شاخه‌ای که **merge** نشده بزن — پیامش رو بخون. بعد با **-D** حذفش کن و با **reflog** کامیت‌هاش رو پیدا کن.
3. سوال: همکاری گفته «**main** رو آپدیت کردم» — تو داری روی **feature/x** کار می‌کنی و می‌خواهی تغییرات جدید **main** رو هم داشته باشی. چیکار می‌کنی؟ (راهنمایی: **merge** یا **rebase** — فصل ۸ تصمیمش رو می‌دیم.)

```
git switch -c temp-work
echo hi > t.txt && git add . && git commit -m "test: temp"
git switch main
git branch -d temp-work
# error: The branch 'temp-work' is not fully merged. ← گیت محافظت می‌کند!
git branch -D temp-work          # حذف اجباری
git reflog | grep temp           # یا: git log --oneline --reflog
git reset --hard <SHA-found>     # نجات کامیت‌ها (یا ساخت شاخه از آنجا)
git switch -c temp-restored
```

→ فصل بعد: rebase و interactive rebase – تاریخچه‌ای که آدم خجالت نمی‌کشد نشونش بده.

فصل ۸ – Rebase و هنر تاریخچه تمیز

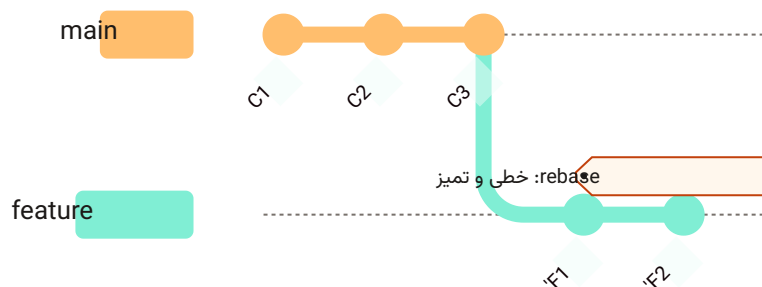
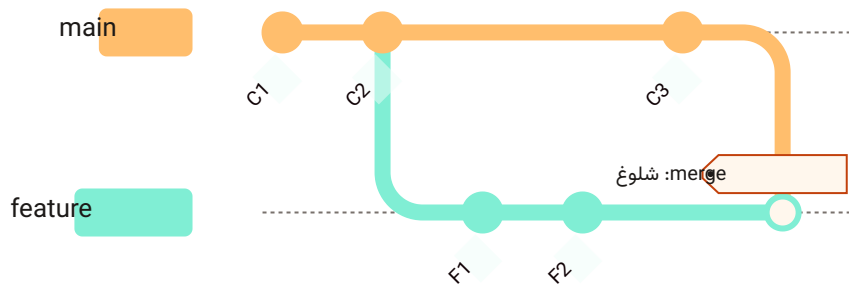
🎯 هدف: rebase – مفهومی که تو مصاحبه‌ها همیشه می‌پرسن و در تیم‌ها هر روز استفاده می‌شه. آخر فصل می‌دونی کی merge و کی rebase، و با interactive rebase کامیت‌های قاطی‌پات رو تمیز می‌کنی.

۸.۱ Rebase چیست؟

مشکل آشنا: داری روی `feature/x` کار می‌کنی؛ همکات main رو جلو برده. تو می‌خوای کارت «روی جدیدترین main» باشه. دو راه:

راه الف – merge (فصل قبل): تغییرات main رو به شاخهت بیار → تاریخچه پر از merge commit های «Merge branch 'main' into feature/x».

راه ب – rebase: کامیت‌های تو رو **بکن** (کندن = pull up) و روی نوک جدید main دوباره بکار → تاریخچه خطی و تمیز، انگار از اول روی main جدید کار می‌کردی:



⚠️ دقت کن: در rebase، کامیت‌های تو (F1, F2) کاملاً جدیدن – SHA های متفاوت با همون محتوا. یعنی rebase = بازنویسی تاریخچه. این نکته، کل بحث ایمنی این فصله.

```
git switch feature/x
git rebase main      # «منتقل کن کامیت‌های شاخه جاری را روی آخرین کامیت شاخه»
```

حل می‌شه، ولی مرحله‌ای merge اگر تعارض پیش اومد - مثل

فایل‌ها رو درست کن ۱.

```
git add file.js
```

```
git rebase --continue      # (برای عقب‌نشینی کامل --abort یا) تا کامیت بعدی برو
```

۸.۲ – قانون طلایی rebase

هیچ‌وقت روی شاخه‌ای که به مخزن مشترک push شده، rebase نزن.

چرا؟ چون rebase کامیت‌ها رو با SHA جدید می‌سازه. اگر بقیه نسخه قدیمی رو pull کرده باشن، تاریخچه‌ها از هم جدا می‌افتن و push تو باید force بشه - یعنی کار بقیه رو زیر پا می‌ذاری.

شاخه	بازنویسی (rebase/amend)؟
feature/x که فقط تو داری و هنوز merge نشده	✓ آزادی کامل
main یا شاخه‌های مشترک/protected	✗ هرگز
کامیت‌های push شده که دیگران ممکنه کشیده باشن	✗ هرگز

به همین دلیل الگوی استاندارد PR اینه:

```
# تاریخچه خودت رو تمیز کن - PR قبل از باز کردن
git rebase -i main      # کامیت‌های خودم را مرتب/ادغام کن
git switch main && git pull # main کن تازه
git switch feature/x
git rebase main          # بنشین main روی تازه‌ترین
git push --force-with-lease # فقط روی شاخه شخصی خودم! (پایین‌تر توضیح)
```

💡 `--force-with-lease` به جای `force` ساده: فقط اگه ریموت همون چیزی باشه که تو دیدی، بازنویسی کن. اگه همکاری وسط راه چیزی push کرده باشه، گیت جلوت رو می‌گیره. عادت کن همیشه `with-lease`.

۸.۳ – Interactive Rebase: کارگاه جراحی تاریخچه

قشنگ‌ترین ابزار گیت. با کامیت‌های نمونه شروع کن:

```
git lg -5
# a1b2c3d (HEAD → feature) feat: add search endpoint
# e4f5g6h fix: typo
# i7j8k9l fix: typo again
# m0n1o2p wip
# p3q4r5s (main) chore: bump deps
```

سه کامیت وسط قشنگ نیستن. جراحی کنیم:

```
git rebase -i p3q4r5s      # یا: git rebase -i HEAD~4
```

ادیتور باز می‌شه با لیست کامیت‌ها از قدیم به جدید:

```
pick m0n1o2p wip
pick i7j8k9l fix: typo again
pick e4f5g6h fix: typo
pick a1b2c3d feat: add search endpoint
```

دستورها رو عوض کن:

کار	دستور
همین‌طور نگه دار	<code>pick</code> / <code>p</code>
نگه دار ولی پیامش رو عوض کن	<code>reword</code> / <code>r</code>
نگه دار ولی محتوایش رو هم ویرایش کنم	<code>edit</code> / <code>e</code>
با کامیت قبلی ادغام (پیام‌ها ترکیب می‌شن)	<code>squash</code> / <code>s</code>
با قبلی ادغام و پیامش دور انداخته شه ★	<code>fixup</code> / <code>f</code>
کاملاً حذف	<code>drop</code> / <code>d</code>
ترتیب کامیت‌ها عوض می‌شه	جابجایی خط‌ها

نسخه تمیزشده:

```
pick m0n1o2p wip
reword m0n1o2p      ← رو درست کن wip پیام
fixup i7j8k9l        ← حل می‌شوند wip ها در typo
fixup e4f5g6h
pick a1b2c3d feat: add search endpoint
```

نتیجه: دو کامیت تمیز به جای چهار کامیت پراکنده. این همون کاریه که قبل از باز کردن PR انجام می‌دی — ریویوچی فقط کار مرتب می‌بینه.

🔔 اگر وسط interactive rebase گیج شدی: `git rebase --abort` – همه چیز به قبل برمی‌گرده (و حتی اگر کامل شد، reflog هست). جراحی تاریخچه = عملی که همیشه دکمه لغو داره.

۸.۴ – cherry-pick: برداشتن یک کامیت خاص

سناریوی واقعی: فیکس حیاتی روی شاخه دیگه‌ست (یا از main باید به شاخه release منتقل شه – backport):

```
git switch release/1.4
git cherry-pick a1b2c3d    # فقط «همون» کامیت را اینجا تکرار کن
# add + cherry-pick --continue : حلش کن merge تعارض هم ممکنه – مثل #
```

۸.۵ – merge یا rebase؟ (جدول تصمیم)

انتخاب	موقعیت
<code>rebase -i</code> (تمیز کردن)	کامیت‌های محلی شاخه ویژگی قبل از ارسال PR
<code>rebase</code> (تاریخچه خطی) – یا <code>merge</code> ، بسته به قرارداد تیم	انتقال آخرین تغییرات شاخه اصلی (main) به شاخه جاری
<code>merge</code> (در گیت‌هاب – فصل ۱۱)	ادغام نهایی PR در main
❌ هیچ بازنویسی‌ای – فقط جلو	کامیت روی شاخه مشترک push شده

💡 تیم‌ها قراردادشون متفاوت: بعضی «merge only»، بعضی «rebase to keep linear». سوال مهم روز اول: «workflow گیت‌تون چیه – merge می‌کنید یا squash؟» این سوال خودش نشون می‌ده تو روحیه تیمی بلدی!

✅ جمع‌بندی فصل

- `rebase` = کندن کامیت‌های خودت و بکار دوباره روی نوک شاخه مبنا → تاریخچه خطی
- `rebase` بازنویسی تاریخچه است – SHA ها عوض می‌شن
- قانون طلایی: فقط روی شاخه‌های شخصی/تا-push نشده؛ هرگز روی مشترک
- `force push` فقط `force-with-lease --` – و فقط روی شاخه خودت
- `squash/fixup/reword/drop` : `rebase -i` – ابزار تمیزکاری قبل از PR
- `cherry-pick` برای نقل یک کامیت مشخص
- تعارض در `rebase` = همان تعارض `merge`، مرحله‌ای: `--continue + add`، و `--abort` همیشه هست

تمرین فصل ۸

1. زمین تمرین: شاخه بزن و ۴ کامیت زشت بزن (wip1 , wip2 , typo fix , wip3)، بعد با `rebase -i` تبدیلش کن به ۲ کامیت معنادار با Conventional Commits. خروجی `git lg` قبل و بعد را ببین.
2. سناریوی کامل PR (مهم‌ترین تمرین دوره!): از main شاخه بزن → ۳ کامیت بزن → به main یک کامیت اضافه کن (شبیه‌سازی همکار) → `rebase main` روی شاخهات → `git push --force-with-lease` (به ریموت خودت – فصل ۱۰ push رو کامل می‌کنه؛ می‌تونی این مرحله رو بعد فصل ۱۰ تکرار کنی).
3. یک کامیت بنویس «fix: currency in invoice» و با `cherry-pick` به شاخه دیگه‌ش ببر.
4. سوال مصاحبه‌ای: فرق `rebase` و `merge` رو در ۳ جمله بگو.

▼ جواب سوال ۴

«هر دو دستور برای ادغام دو شاخه به کار می‌روند. دستور `merge` با ساخت یک کامیت ادغام جدید (Merge Commit) با دو والد، تاریخچه واقعی انشعاب را مستند می‌کند. دستور `rebase` کامیت‌های شاخه جاری را با شناسه‌های SHA جدید مستقیماً روی نوک شاخه مبنا بازسازی کرده و تاریخچه‌ای کاملاً خطی و یکدست می‌سازد – به همین دلیل `rebase` فقط روی شاخه‌های محلی امن است و نباید روی شاخه‌های عمومی اعمال شود.»

→ فصل بعد: tag، stash و gitignore – ابزارهای کیفیت زندگی روزمره.

فصل ۹ – Stash، Tag و gitignore: ابزارهای کیفیت زندگی

🎯 هدف: سه ابزاری که هر روز کار تو رو راحت تر می‌کنن: نگه‌داشتن کار نیمه‌تمام (stash)، نشونه‌گذاری نسخه‌ها (tag)، و بی‌خیال شدن فایل‌های محلی (gitignore).

۹.۱ – Stash: کمد موقت تغییرات 🧥

سناریوهای واقعی: - داری روی feature کار می‌کنی؛ باگ urgent گزارش می‌شه - کدهای نیمه‌تمامت رو نه کامیت می‌خوام نه حذف - می‌خوای git pull بزنی ولی ویرایش‌های محلی جلوی merge رو می‌گیرن - می‌خوای فقط یه فایل رو ببینی چطوری «قبل از تغییرات» بود

```
git stash                # برو توی کمد (staged + unstaged) تغییرات
git stash -u            # هم بیا (خیلی وقت‌ها لازمه!) untracked فایل‌های
#.. تازه مثل آخرین کامیت - هر کاری بخوای بکن working directory حالا
git stash pop           # و از لیست حذفش کن stash برگردون آخرین
git stash apply         # برگردون ولی در لیست بمونه
git stash list          # کمدها: stash@{0}, stash@{1}, ...
git stash pop stash@{2} # یک کمد خاص
git stash drop stash@{0} # stash دور انداختن یک
```

⚠️ stash یک کامیت واقعی که «قلم‌زده» شده - پس گم‌شدنش هم با reflow برطرف می‌شه. ولی سبک ذهنی درست: stash = کوتاه‌مدت (چند ساعت/یک روز)؛ کار طولانی = شاخه.

الگوی پرکاربرد در شرکت:

```
git stash -u
git switch main && git pull
git switch -c hotfix/critical
# ... کامیت، push، PR ...
git switch feature/my-work
git stash pop
```

۹.۲ – Tag: نشان نسخه‌ها 🏷️

کامیت‌ها زنجیره‌ان؛ tag یعنی «این کامیت، نسخه رسمی ۱.۲.۰ هست». دو نوع:

```

git tag                                # لیست تگ‌ها
git tag v1.0.0                        # سبک: فقط اشاره‌گر
git tag -a v1.0.0 -m "سنگین" # پیام+نام+تاریخ (annotated): اولین ریلیز پایدار"

git show v1.0.0                       # جزئیات تگ
git push origin v1.0.0                # معمولی نمی‌روند push تگ‌ها با ⚠️
git push origin --tags                # یا همه را بفرست
git switch v1.0.0                     # (فصل ۱۳ - detached) سفر به وضعیت آن نسخه

```

Semantic Versioning (قواعد شماره نسخه)

<MAJOR>.<MINOR>.<PATCH> - مثلاً v2.3.1 :

مثال	بالا می‌ره وقتی	جزء
2.0.0	تغییر شکست‌دهنده (API قبلی می‌شکنه)	MAJOR
2.1.0	فیچر جدید سازگار	MINOR
2.1.1	فیکس باگ	PATCH

در گیت‌هاب، از یک تگ یک **Release** می‌سازی (صفحه Releases → Draft a new release) با release notes - جایی که نسخه‌های محصولت رسمی می‌شن.

۹.۳ - gitignore: فایل‌هایی که گیت نبینه 🙈

```

# .gitignore (در ریشه مخزن)
node_modules/      # نصب‌شونده‌ها - هیچ‌وقت
.env               # رمزها و کلیدها - هیچ‌وقت ⭐
dist/              # build خروجی
.next/
*.log              # DS_Store
coverage/
.vscode/           # (یا با جزئیات) تنظیمات شخصی ادیتور

```

الگوها:

الگو	معنی
<code>*.log</code>	همه log ها در همه زیرپوشه‌ها
<code>/dist</code>	فقط dist در ریشه
<code>build/</code>	هر پوشه build در هر عمقی
<code>!important.log</code>	استثنا – این را ignore نکن
<code>debug?.js</code>	? = یک کاراکتر دلخواه
<code>**/temp/*</code>	هر پوشه temp در هر عمقی

🔔 **قانون امنیتی #1 گیت: `.env` و کلید API و پسورد هرگز به مخزن نمی‌رن** – حتی مخزن خصوصی! (فصل ۱۳: اگر رفت، چیکار کنی). پیشنهاد: نمونه امن بذار: `.env.example`. با مقادیر ساختگی `commit` می‌شه و بقیه کپی می‌کنن.

الگوی مفید: فایل‌هایی که `sراسری` می‌خوای `ignore` بشن (برای خودت، نه تیم)

```
git config --global core.excludesfile ~/.gitignore_global
# و ..DS_Store, Thumbs.db در آن فایل
```

نکته‌ای که همه یک بار گیرش می‌افتن: `ignore` کردن فایل‌ی که قبلاً `commit` شده!

`gitignore` فقط فایل‌های `untracked` رو نادیده می‌گیره. فایل `tracked` رو باید اول از ایندکس دربیاری:

```
git rm --cached .env          # از مخزن حذف، از دیسک نمی‌بره
echo ".env" >> .gitignore
git commit -m "chore: remove .env from tracking"
```

۹.۴ – `gitattributes` (یادآوری) و `bisect` (پاداش ویژه 🏆)

- `gitattributes`. فصل ۲ دیدی – قوانین فایل‌ها (eol, binary).
- پاداش: باگ رو نمی‌دونی کدوم کامیت معرفی کرده؟ جستجوی دودویی خودکار:

```
git bisect start
git bisect bad          # الان باگ داره
git bisect good v1.0.0  # این نسخه سالم بود
# گیت وسط می‌رود؛ تست می‌کنی و می‌گویی:
git bisect good         # تا کامیت مقصر پیدا شود - bad یا
git bisect reset
```

با ۱۰۰۰ کامیت، فقط ~۱۰ تست لازمه. ابزار «کارمنده که شب‌ها برات کار می‌کنه».

✓ جمع‌بندی فصل

- `stash -u` کار نیمه‌تمام رو موقتاً کنار می‌ذاره؛ `pop` برمی‌گردونه؛ برای کارهای بلند شاخه بزن نه `stash`
- تگ `annotated` برای نسخه‌ها؛ **تگ‌ها جدا push می‌شن**؛ `semver: MAJOR.MINOR.PATCH`
- `.gitignore` همیشه از روز اول: `.env` ، `node_modules/` ، خروجی `build`؛ فایل `tracked` باید با `rm --cached` آزاد شه
- `bisect` = جستجوی دودویی باگ در تاریخچه

📝 تمرین فصل ۹

1. زمین تمرین: تغییراتی در دو فایل بده + یک فایل جدید بساز؛ `stash` بزن (بدون `-u`) و بعد `pop` – فایل جدید برگشت؟ چرا؟ بعد با `-u` تکرار کن.
2. برای زمین تمرین سه تگ بزن (`v0.1.0` و ...) و سفر کن: `git switch v0.1.0` – پیام `HEAD` چی می‌گه؟ (`git switch main` برای برگشت!)
3. یک `.gitignore` کامل برای پروژه `Next.js` بنویس (بدون نگاه به جواب).
4. (چالش) با `bisect`: در زمین تمرین ۱۰ کامیت بزن که در کامیت ۷ یک باگ «معرفی» شده؛ بعد با `bisect` پیداش کن.

▼ جواب تمرین ۳

```
# .gitignore برای Next.js
node_modules/
.next/
out/
build/
.env*.local
.env
.vercel
*.tsbuildinfo
next-env.d.ts      (می‌شود ignore نسل جدید خودکار تولید می‌شود - بحث‌برانگیز، معمولاً)
.DS_Store
npm-debug.log*
```

جواب تمرین ۱: نه – بدون `-u` ، فایل `untracked` در `stash` نمی‌آید و سرجاش می‌ماند.

➔ **فصل بعد:** بالاخره وارد گیت‌هاب عمیق می‌شویم – ریموت‌ها، `push/pull/fetch` و تفاوت‌های ظریفشون.

فصل ۱۰ – گیت‌هاب و ریموت‌ها: push، pull، fetch و فورک

🎯 هدف: رابطه مخزن لوکالت با گیت‌هاب رو شفاف کنی: origin/upstream چین؟ فرق pull و fetch؟ فورک چیه و کی لازمه؟ آخر فصل، چرخه کامل کار با مخزن شرکتی رو بلدی.

۱۰.۱ – Remote: آدرس مخزن در دنیای بیرون

مخزن لوکالت یک ریموت داره: آدرس نسخه ابری. ببینش:

```
git remote -v
# origin git@github.com:username/my-project.git (fetch)
# origin git@github.com:username/my-project.git (push)
```

- **origin** = اسم قراردادی ریموت «خانه اصلی پروژه» (می‌تونه هر اسمی باشه؛ ولی origin استاندارد)
 - یک مخزن می‌تونه چند ریموت داشته باشه (مثلاً origin = فورک خودت، upstream = مخزن شرکت)
- شاخه‌های ریموت هم در لوکال شما آینه می‌شن:

```
git branch -a
# * feature/x
# main
# remotes/origin/main      ← تازه می‌شود fetch فقط با آینه آخرین وضعیت ریموت
# remotes/origin/HEAD
```

🔑 مفهوم کلیدی: **origin/main** یک کپی محلی از وضعیت ریموت در لحظه آخرین **fetch** است – نه وضعیت زنده! تا **fetch** نکنی، گیت چیزی از دنیای بیرون نمی‌دونه. همه کارهای گیت لوکال، آفلاین.

۱۰.۲ – fetch در مقابل pull: فرقی که در شرکت مهمه

دستور	کار	کی
<code>git fetch</code>	فقط <i>دانلود</i> وضعیت جدید ریموت به <code>origin/*</code> – کدت دست نمی‌خوره	می‌خوام اول ببینم چی اومده
<code>git pull</code>	<code>fetch + merge</code> (یا <code>rebase</code>) در شاخه فعلی	«همه‌چیز رو بیار و ادغام کن»

```
git fetch origin
git lg --all          # حالا ببین همکارت چه آورده
git diff main origin/main # تفاوت محلی با شاخه ریموت چیست؟
git merge origin/main  # تصمیم آگاهانه – (یا rebase)
```

💡 توصیه حرفه‌ای: عادت کن **fetch** + نگاه + بعد ادغام. **pull** کورکورانه دقیقاً همون چیزیه که **merge conflict** های غافلگیرکننده می‌سازه. (اگه **pull** می‌زنی، حرفه‌ای‌ترش: **git pull --rebase** که تاریخچه رو خطی نگه می‌داره — فصل ۸.)

۱۰.۳ — push و upstream

`git push` چی می‌کنه؟ *hood* کاری که بلدی — ولی زیر #

اولین **push** یک شاخه جدید (میان‌بر مدرن — از Git 2.37 بدون تنظیم اضافه کار می‌کنه):

```
git push -u origin feature/x
# -u (upstream): این شاخه لوکال را «جفت» کن با origin/feature/x
# git push: بعد از آن، در همان شاخه فقط
```

چند واقعیت **push**:

- **push** فقط کامیت‌های جدید رو می‌فرسته (snapshot های قبلی از قبل رفته‌ان)
- **push** پیش‌فرض **fast-forward** فقط: اگه ریموت جلوتر از تو باشه، رد می‌شه: **[rejected] main → main** !
- (**fetch first**) علاج درست: **git pull --rebase** بعد **push** — هرگز از اول **force** نزن!
- بازنویسی تاریخچه (**rebase/amend** روی شاخه شخصی **push** شده) → **git push --force-with-lease**

```
git push --delete origin feature/old      حذف شاخه از ریموت #
git push origin v1.2.0                      پوش تگ‌ها (فصل ۹) #
```

۱۰.۴ — clone، fork و روابطشون

سه راه ورود به یک مخزن گیت‌هاب:

روش	کی	در شرکت
clone مستقیم مخزن	عضو تیم/organization هستی با دسترسی write ★	۹۵٪ موارد
fork + clone	پروژه open source ای که maintainer اش نیستی	contribution به پروژه‌های خارجی
دانلود ZIP	فقط نگاه	(گیت نداره! رشدت رو نشون نده 😊)

clone با fork فرق

- **clone** = کپی لوکال از مخزن (توی سیستم)
- **fork** = کپی کامل مخزن در اکانت گیت‌هاب خودت (روی ابر)

```
# ۱. ساختن می‌شود Fork → github.com/your-username/cool-lib
git clone git@github.com:your-username/cool-lib.git
cd cool-lib

# ۲. ریموت مخزن اصلی را اضافه کن (برای همگام موندن):
git remote add upstream git@github.com:original/cool-lib.git
git remote -v
# origin    git@github.com:your-username/cool-lib.git  (مقصد - push مخزن فورک شده شخصی)
# upstream  git@github.com:original/cool-lib.git      (فقط - fetch اصلی)

# ۳. همیشه قبل از شروع کار، از منبع اصلی تازه کن:
git fetch upstream
git switch main && git merge upstream/main  # یا rebase
git push origin main

# ۴. (فورک خودت) origin به push، شاخه بزن، کار کن
# بزن (فصل ۱۱) Pull Request و از گیت‌هاب به مخزن اصلی
```

۱۰.۵ – ساخت مخزن و حرفه‌ای‌سازی پروژه‌ها

روی گیت‌هاب: **New repository** → اسم، عمومی/خصوصی، تیک **Add a README**. بعد از لوکال:

```
git remote add origin git@github.com:your-username/new-project.git
git push -u origin main
```

چند تکلیف یک‌بار-مصرف که پروژه‌ها رو از «تمرینی» به «حرفه‌ای» تبدیل می‌کنه:

فایل	چرا
<code>README.md</code> خوب	اولین چیزی که مصاحبه‌گر می‌بینه: چی، چرا، چطور اجرا شه
<code>.gitignore</code> از روز اول	<code>node_modules</code> و <code>env</code> . نیز بالا (فصل ۹)
توضیحات (About) + topics	قابلیت جستجو در گیت‌هاب
لایسنس (MIT و ...)	پروژه‌ی open source بدون لایسنس = «همه حق محفوظ» به صورت پیش‌فرض

۱۰.۶ – گردش کامل یک روز کاری در شرکت

```
git switch main && git pull          # صبح: تازه‌سازی
git switch -c fix/invoice-total      # شاخه کار
#.. کد، تست ..
git add -p && git commit -m "fix(invoice): round totals correctly"
git fetch origin                    # تکنون خورده؟ main ببین
git rebase origin/main              # تاریخچه خطی (اختیاری)
git push -u origin fix/invoice-total # اولین پوش شاخه
# (!فصل ۱۱) Compare & pull request: روی گیت‌هاب →
```

جمع‌بندی فصل ✓

- `origin` = آدرس مخزن ابری؛ `origin/main` = آینه لوکال ریموت (فقط با `fetch` تازه می‌شه)
- `fetch` = فقط دانلود؛ `fetch` = `pull` + ادغام. عادت حرفه‌ای: `fetch` → نگاه → ادغام (`--rebase`)
- `push -u` جفت‌سازی شاخه لوکال با ریموت؛ رد شدن `push` یعنی «اول `--rebase pull` کن» – نه `force`!
- `fork` = کپی مخزن در اکانتت برای `contribution` به پروژه‌های خارجی؛ داخل شرکت معمولاً `clone` مستقیم + شاخه
- 1-day شرکت: `fetch` → `clone` → `SSH` → شاخه جدید

تمرین فصل ۱۰ 📝

1. در یکی از پروژه‌های خود دستور `git remote -v` را اجرا کنید – `SSH` هست یا `HTTPS`؟ (اگه `HTTPS` با رمز کار می‌کنی، مهاجرتش کن به `SSH` – با `git remote set-url`)
2. روی گیت‌هاب یک مخزن جدید خالی بساز و از لوکال بهش `push` کن – چرخه کامل `push -u` → `remote add`.
3. یک پروژه محبوب `open source` رو `fork` کن، `clone` اش کن، `upstream` رو `add` کن و `git fetch upstream` بزن. (بدون تغییری – فقط زیرساخت `contribution` رو آماده کنی.)
4. `git fetch` بزن و بعد `git diff main origin/main` – فرق لوکالت با ریموت چیه؟

▼ راهنمای تمرین ۱

```
git remote -v
git remote set-url origin git@github.com:USER/REPO.git  # مهاجرت به SSH
ssh -T git@github.com                                     # تست
git fetch && git push                                       # تأیید نهایی
```

➔ فصل بعد: Pull Request – جایی که همکاری واقعی گیت‌هابی شکل می‌گیره.

فصل ۱۱ – Pull Request: قلب همکاری در گیت‌هاب

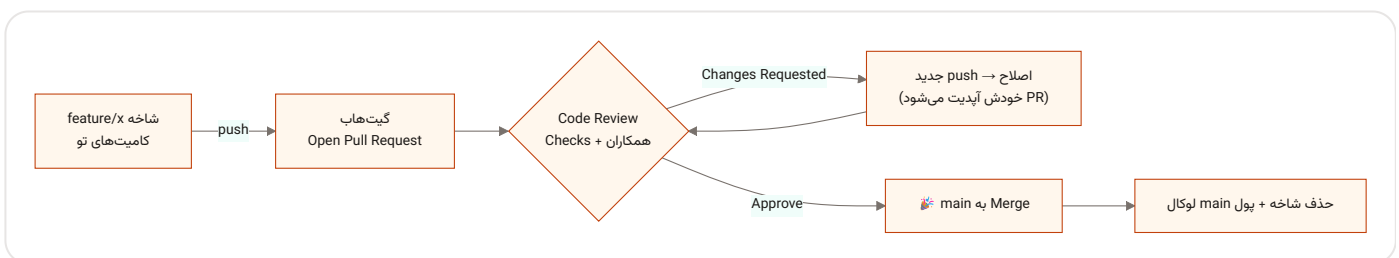
🎯 هدف: PR رو از «دکمه‌ای که می‌زنم» به «فرایند حرفه‌ای بازبینی و ادغام» تبدیل کنی: چطور PR خوب بنویسی، ریویو بگیری و ریویو بدی، و سه استراتژی merge گیت‌هاب چه فرقی دارن. این فصل، مهم‌ترین مهارت تو برای کار در شرکت است.

۱۱.۱ PR اصلاً چیست؟

Pull Request = درخواست «کدم رو به main اضافه کنید». یک صفحه گفتگو درباره شاخه‌ت که شامل:

- **diff کامل** تغییرات تو نسبت به main
- توضیحات تو (چرا؟ چطور؟ چطور تست شد؟)
- **ریویو همکاران** (کامنت، درخواست تغییر، تأیید)
- **Checks** – تست‌های خودکار (فصل ۱۵)
- دکمه نهایی: **Merge**

نامش از قدیم مانده: «درخواستِ pull» – یعنی از maintainer می‌خوای کدت رو pull کنه. (گیت‌هاب رابطش رو «Pull Request» می‌نامد؛ گیت‌لب: Merge Request – همون چیزیه.)



💡 نکته‌ای که خیلی‌ها نمی‌دونن: PR شاخه رو دنبال می‌کنه، نه snapshot. بعد از باز شدن PR، هر push جدید به همون شاخه، خودکار به PR اضافه می‌شه – برای اصلاح نیازی به PR جدید نیست!

۱۱.۲ چرخه کامل: از کد تا merge

```
# شده push شاخهات (از فصل ۱۰):  
git push -u origin fix/invoice-total
```

حالا روی گیت‌هاب: 1. بالای صفحه بنر زرد **Compare & pull request** ظاهر می‌شه (یا: تب New → Pull requests → Pull requests → New) انتخاب 2 (compare: `fix/invoice-total` ← base: `main`). **عنوان و توضیحات** بنویس (بخش ۱۱.۳) 3. **Reviewers** رو تعیین کن (اگه CODEOWNERS تنظیم شده باشه خودکار میاد - پایین‌تر) 4. Submit → منتظر ریویو و سبز شدن Checks

ریویو اومد - «Changes requested»

عادی‌ترین اتفاق دنیااست؛ ناراحت نشو، این یعنی ریویو واقعی انجام شده:

```
# اصلاح کن (روی همان شاخه!):
git switch fix/invoice-total
# ... تغییرات...
git add -p && git commit -m "fix: address review - handle null coupon"
git push                # ← PR نیست چیزی لازم می‌شود؛ چیزی لازم نیست PR ←
# بزن Resolve بده و Reply به همکاری در کامنت‌ها
```

بعد از merge:

```
git switch main
git pull
git branch -d fix/invoice-total
git push --delete origin fix/invoice-total # در گیت‌هاب Delete branch با دکمه
```

۱۱.۳ - PR خوب بنویس (هنر نیمه‌ی ماجراست)

ریویوچی وقت نداره. PR تو باید در ۳۰ ثانیه «چرا و چی» رو برسونه:

قانون‌ها: - کوچک! ~۲۰۰-۴۰۰ خط diff ماکس. PR بزرگ = ریویوی سطحی یا هیچ. کار بزرگ رو به چند PR بشکن. - یک موضوع - قاطی نکن فیکس و ریفاکتور و فیچر - خودت رو جای ریویوچی بذار: «بدون توضیح تو، متوجه می‌شه؟»

قالب توضیحات (همینو کپی کن):

```
## چه چیزی
.فیکس گرد کردن مبلغ کل فاکتور در حالت اعشار (ای شو #142)

## چرا
ساده گرد می‌شد و در پرداخت‌های تقسیمی round با total تا حالا
تا ۱ تومان اختلاف تولید می‌کرد (اسکرین‌شات در تیکت)

## چطور تست شد
✓ (جدید case ۴) `invoice-total.spec.ts` تست واحد جدید
- با سفارش واقعی تقسیمی staging تست دستی روی محیط

## ریسک/نکته برای ریویو
رفتارش کمی عوض شده - فایل‌های `billing.ts` در `calcTotal` متد
هم مصرف‌کننده‌ان، نگاهی بندازید `payment.ts` و `invoice.ts`
```

عنوان PR = پیام کامیت اصلی: `fix(invoice): round totals correctly (closes #142)`

۱۱.۴ – سمت دیگر: ریویو دادن

تو هم زودتر از فکر در موقعیت ریویو دهنده قرار می‌گیری. آدایش:

برای نویسنده (از دید ریویوچی): | بایست | نبایست | |---| | روی کد نظر بده نه آدم | «چرا اینو اینجوری نوشتی؟!» با لحن حمله | | سوال بپرس: «اینجا اگر coupon=null باشه چی می‌شه؟» | تحمیل سلیقه شخصی بدون دلیل فنی («من همیشه این مدلی می‌نویسم») | | مشخص کن: Nit (جزئی) یا Blocking (الزامی) | «LGTМ» بدون خوندن | | در ۲۴ ساعت جواب بده | هفته‌ها نگه داری و PR استیل شه |

💡 قاعده طلایی ریویو: «سخت‌گیر روی کد، مهربان با آدم.» و به عنوان نویسنده: ریویو خوردن = gift، نه حمله. بهترین دولوپرها بیشتر از همه ریویو می‌گیرن چون PR های بیشتری می‌زنن.

کامنت‌های گیت‌هاب با `suggestion block` امکان اعمال یک‌کلیکی پیشنهاد رو می‌دن – ارزشون استفاده کن وقتی ریویو می‌دی.

۱۱.۵ – سه استراتژی Merge گیت‌هاب

وقتی PR تأیید شد، دکمه merge سبز می‌شه – ولی گیت‌هاب سه تا دکمه داره (طبق تنظیمات مخزن):

دکمه	کاری که می‌کنه	تاریخچه حاصل	مناسب
Create a merge commit	کامیت merge با همه کامیت‌های شاخه	درخت واقعی + گراف شلوغ	تیم‌هایی که تاریخچه واقعی می‌خوان
Squash and merge	همه کامیت‌های شاخه → یک کامیت تمیز در main	main خطی و خوانا	اکثر تیم‌های مدرن
Rebase and merge	کامیت‌ها یکی‌یکی روی main (بدون کامیت merge)	خطی، ولی کامیت‌های wip هم می‌مونن	شاخه‌هایی که خودشون تمیز بودن

با squash، تمام `wip` ها و typo-fix های تو به یک کامیت: `fix(invoice): round totals correctly (#201)` تبدیل می‌شه. به همین دلیل خیلی تیم‌ها «شاخه آزادانه کار کن، main همیشه تمیز» رو با squash پیاده می‌کنن – و در فصل ۸ گفتیم قانون طلایی rebase هم با این مدل سازگارتره.

سوال روز اول شرکت: «PR ها رو squash merge می‌کنید یا merge commit؟» جوابش، کل تاریخچه‌ی آینده مخزن رو تعیین می‌کنه.

۱۱.۶ – حفاظت‌ها: Rulesets و CODEOWNERS

main در شرکت‌ها **protected** است – نمی‌تونی مستقیم push کنی. ابزار مدرن گیت‌هاب: **Rulesets** (جایگزین جدید Branch Protection):

- مجاز نبودن push مستقیم به main (همه‌چیز از PR بگذره)
- الزام حداقل ۱-۲ Approval
- الزام سبز بودن Checks (تست‌ها)
- الزام شاخه به‌روز با main قبل merge

و **CODEOWNERS** – فایل‌ی که تعیین می‌کنه ریویو کی الزاوریه:

```
# .github/CODEOWNERS
*                @team-backend      # مخزن: تیم بک
/src/ui/         @team-frontend    # تیم فرانت: UI مسیر
/payment/       @security-lead     # پول = چشم لید امنیت
```

وقتی فایل‌ی از این مسیرها در PR تغییر کنه، owner هاش خودکار reviewer می‌شن و بدون approve شون merge نمی‌شه.

۱۱.۷ – Issue: برگه‌ی کارها، باگ‌ها و گفتگوها

قبل از کد، همیشه یک «قرارداد» وجود داره: قراره دقیقاً چی حل بشه؟ در گیت‌هاب این قرارداد **Issue** نام داره – لیست کارهای مخزن:

- **باگ گزارش کن** («با کد تخفیف **SALE50**، سبد خرید عدد منفی می‌شه») – با قدم‌های بازتولید و اسکرین‌شات
- **فیچر پیشنهاد بده**، سوال بپرس، برای هر کار یک Issue جدا
- هر Issue یک شماره می‌گیره: **#142** – این شماره در کل گیت‌هاب آدرس‌پذیره (هر جا بنویسی **#142** لینک می‌شه)
- با **Labels** دسته‌بندی کن (**Assignee**)، (**good first issue**، **enhancement**، **bug** مشخص کن، و در **Projects** مورد کنترل (کانبان) بچین

اتصال Issue به کامیت و PR – جایی که گیت‌هاب جادویی می‌شه

در پیام کامیت یا توضیحات PR این کلیدواژه‌ها رو بنویس و شماره Issue رو بده:

```
git commit -m "fix(cart): clamp total to zero when coupon exceeds amount
Fixes #142"
```

کلیدواژه	اثر بعد از merge شدن PR به شاخه پیش فرض
Fixes #142 / Closes #142 / Resolves #142	Issue خودکار بسته می‌شه و به PR لینک می‌شه
Refs #142 / #142	فقط لینک و ردیابی – بسته نمی‌شه

از این به بعد «کد بدون شرح» نداری: هر تغییر به یک Issue وصله، تاریخچه‌ی Issue هم به کامیت‌ها و PR مربوطه لینک داره – کل ماجرا از «چرا این کد اینجوریه؟» تا «کی فیکس شد؟» یک کلیک فاصله داره.

💡 شروع اوپن سورس: مخزن‌های بزرگ برای تازه‌واردها Label با اسم `good first issue` می‌ذارن. یک Issue آسون بردار، فورک کن (فصل ۱۰)، فیکس کن، PR بزن و در توضیحاتش بنویس `Fixes #123` – اولین مشارکت جهانی‌ات همین‌قدر نزدیکه.

جمع‌بندی فصل ✓

- PR = درخواست ادغام + گفتگوی ریویو + checks؛ **شاخه رو دنبال می‌کنه** – push جدید = آپدیت خودکار
- PR خوب: کوچک، تک‌موضوع، با قالب «چه/چرا/چطور تست شد»
- ریویو: سخت روی کد، مهربان با آدم؛ به عنوان نویسنده، ریویو = هدیه
- سه merge گیت‌هاب: `squash` / `rebase` / `merge commit` – بپرس تیم کدوم رو می‌خواد
- main معمولاً با Rulesets محافظت می‌شه؛ CODEOWNERS ریویوی الزامی رو خودکار می‌کنه
- Issue = لیست کارهای مخزن؛ `Fixes #142` در کامیت/PR یعنی بعد از merge، ایشو خودکار بسته می‌شه
- بعد از merge: پول main، حذف شاخه، و شروع تمیز بعدی

تمرین فصل ۱۱ (کامل‌ترین تمرین دوره – حتماً انجامش بده!) 📝

۱. روی یکی از پروژه‌های خودت یک بهبود کوچیک واقعی پیدا کن (مستندات، refactor کوچیک، فیکس).
۲. شاخه بزن، ۲-۳ کامیت بزن (حتی یکی wip)، push کن، و PR با قالب بخش ۱۱.۳ باز کن.
۳. **خودت ریویو بده:** در تب Files changed، حداقل ۳ کامنت واقعی بذار (یکی suggestion block).
۴. اصلاح کن، push کن، Resolve کن، و با **Squash and merge** ادغامش کن.
۵. در توضیحات PR یک Issue هم بساز (`Fixes #1`) و ببین بعد از merge، ایشو خودکار بسته می‌شه.
۶. برگرد لوکال: pull، حذف شاخه، `git lg` – به تمیزی main نگاه کن. این چرخه رو تا وقتی «عادی» حسش کنی روی پروژه‌های دیگه تکرار کن.

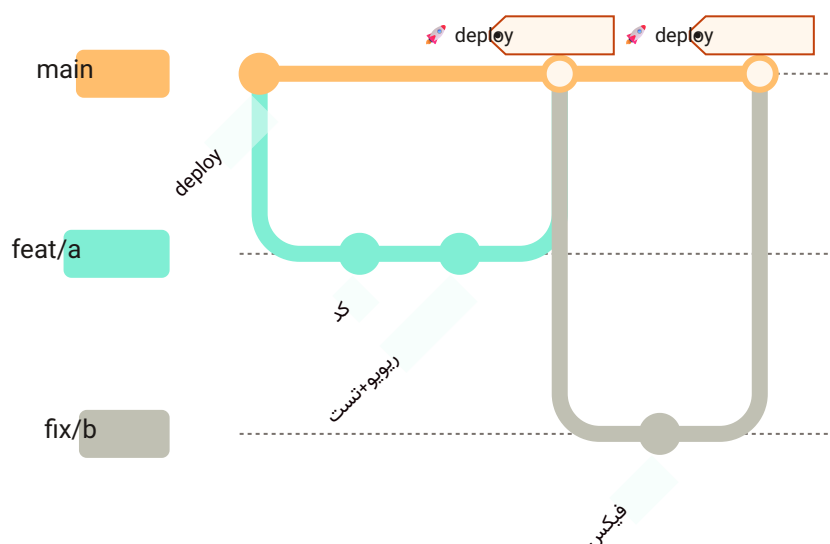
➡ **فصل بعد:** گردش کارهای تیمی – Git Flow، GitHub Flow، و trunk-based؛ و اینکه هر شرکت کدوم رو می‌خواد.

فصل ۱۲ – گردش کارهای تیمی: وقتی «تیم» وارد ماجرا می‌شود

🎯 هدف: در شرکت‌ها، گیت «هر کی هر جور دوست داره» وجود نداره – یک workflow قراردادی هست. سه گردش کار استاندارد صنعت رو یاد می‌گیری تا روز اول استخدام بدونی پیش روته، و برای تیمت هم بتونی انتخاب کنی.

۱۲.۱ – GitHub Flow: ساده و مدرن ★

ساده‌ترین گردش کاری که اکثر استارت‌آپ‌ها و محصولات وب مدرن (خود GitHub هم!) استفاده می‌کنن:

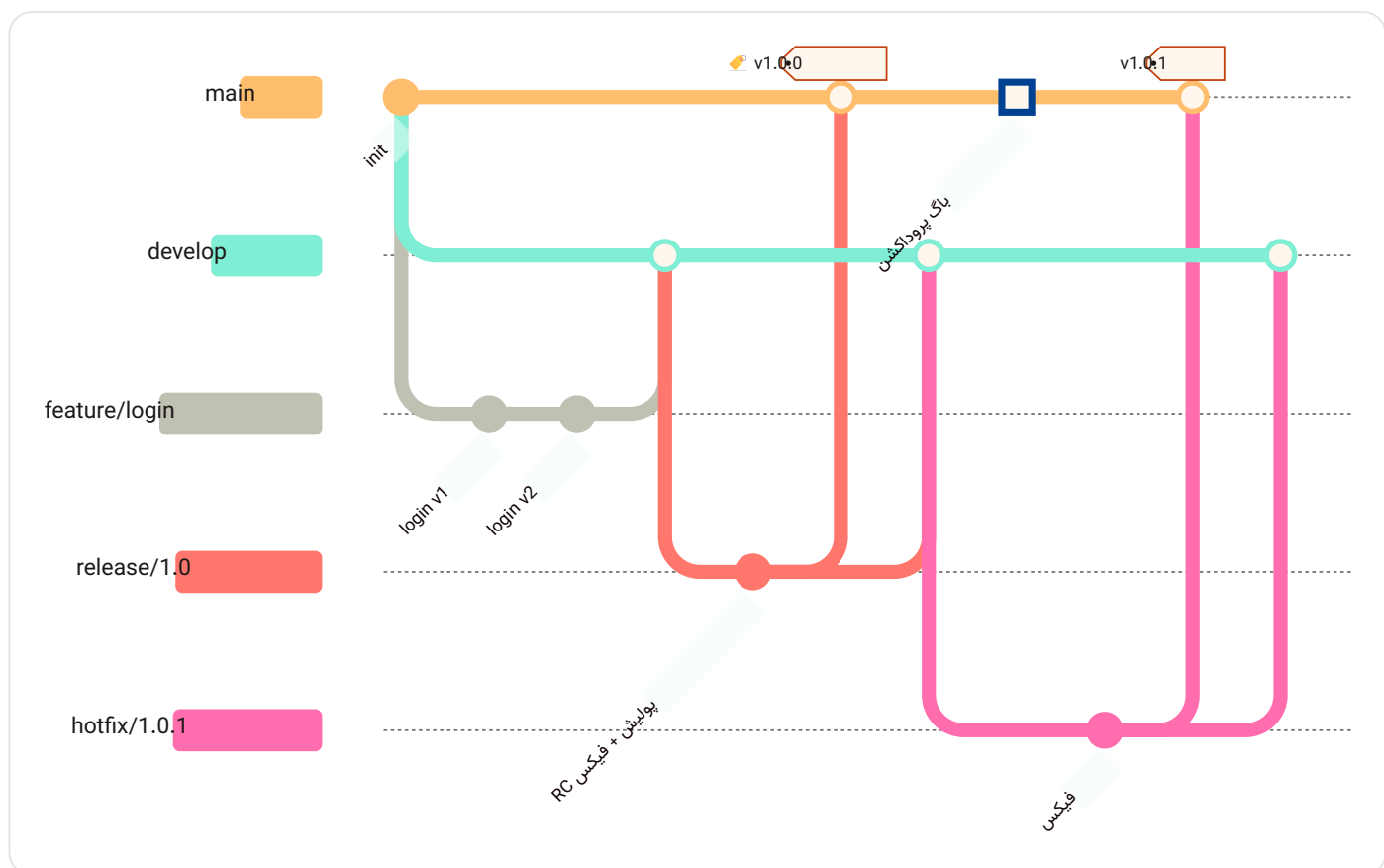


قوانین (فقط ۵ تا!): 1. **main** همیشه **deploy-پذیر** است – هر کامیتش قابل انتشار 2. هر کار جدید = شاخه جدید از 3. **main** کامیت‌ها + **push** منظم (نه «تموم شد بعد push») **Pull Request** 4. دروازه merge (ریویو + checks) 5. بعد از **deploy** → **merge** → شاخه پاک

- ✓ مزیت: ساده، سریع، کاملاً PR-محور – برای CI/CD مداوم عالیه
 - ✗ ضعف: نسخه‌های موازی (مثلاً «نسخه موبایل X.۱ و X.۲ باهم پشتیبانی می‌شن») رو سخت می‌کنه
- برای چه کسی: SaaS، وب‌اپ، هر جا **deploy** پیوسته داره → انتخاب پیش‌فرض مدرن.

۱۲.۲ – Git Flow: سنگین‌تر، برای ریلیزهای برنامه‌ریزی شده

گردش کار کلاسیک با دو شاخه دائمی و چند شاخه موقت:



شاخه	دائمی؟	نقش
main	✓	فقط نسخه‌های منتشرشده – هر کامیت یک tag
develop	✓	حلقه ادغام فیچرها
feature/*	موقت	کار جدید از develop
release/*	موقت	آماده‌سازی نسخه: پولیش، bump نسخه، فیکس RC
hotfix/*	موقت	فیکس فوری از main → برگشت به main و develop

• ☒ مزیت: مدیریت چند نسخه موازی، ریلیزهای برنامه‌ریزی‌شده – برای اپ موبایل/دسکتاپ که نسخه‌ها «زنده» می‌مونن

• ☒ ضعف: سنگین و پرآیین – برای وب‌سایت‌های معمولی overkill

برای چه کسی: اپ‌های با ریلیز نسخه‌ای (موبایل، دسکتاپ، embed)، تیم‌های بزرگ با چرخه release مشخص.

۱۲.۳ – Trunk-Based Development: سبک شرکت‌های بزرگ

سبک غالب شرکت‌های مقیاس‌بزرگ (Google و مشابه‌ها): همه به یک trunk (main) خیلی کوچک و سریع کامیت می‌زنن:

- شاخه‌ها کوتاه‌عمر (چند ساعت/چند روز max)، ادغام‌های مکرر
- فیچر ناتمام؟ **Feature Flags** – کد merge می‌شه ولی با فلگ خاموش deploy می‌شه
- به جای شاخه‌های بلند و merge های فاجعه‌بار، ادغام‌های مکرر کوچک

```
# سریع merge، شاخه کوتاه:
git switch -c quick-fix && # ... ۱-۲ کامیت..
git push && gh pr create --fill # فوری PR
# فیچر بزرگ؟ تکه‌تکه (مثل برگ)، نه یک ماموت
# یک تکه قابل‌ریویو که با فلگ خاموشه = PR هر
```

- مزیت: ادغام مداوم = تعارض‌های کوچک و نادر؛ سرعت تیم‌های بزرگ ✓
- نیازمند بلوغ: تست‌های خودکار قوی، CI خوب، فرهنگ ریویو سریع ✗

۱۲.۴ – جدول انتخاب (و اینکه در واقعیت با چی روبه‌رو می‌شی)

Trunk-Based	Git Flow	GitHub Flow	
main	main + develop	main	شاخه‌های دائمی
کوتاه‌عمر + فلگ	از develop، عمر بلندتر	از main، عمر متوسط	شاخه‌ی کار
مداوم (چند بار در روز)	با tag ریلیز	هر merge	deploy
تیم بزرگ/بلوغ بالا	اپ نسخه‌ای	وب/SaaS ★	مناسب
متوسط (نیاز به زیرساخت)	زیاد	کم ★	پیچیدگی

💡 **واقعیت بازار کار:** اکثر شرکت‌های ایرانی و استارت‌آپ‌ها **GitHub Flow + squash merge** دارند. اگه بپرسی «**workflow** گیت‌تون چیه؟» و جوابش روشن بود، یعنی تیم آگاهی؛ اگه نه، تو با دانش این فصل می‌تونی کمکی کنی – و این نشونه‌ی بلوغه.

۱۲.۵ – عرف‌های تیمی که باید از شون باخبر شی (چک‌لیست روز اول)

- PR؟ و فقط protected می‌شه با push مستقیم main
- merge: squash / merge commit / rebase-and-merge؟ استراتژی
- قرارداد نام شاخه: feature/*، fix/*، user/initials؟
- لینک به تیکت (JIRA/Trello)؟
- CODEOWNERS لازمه؟ Approve حداقل چند
- lint/test/build الزامی چنان Checks؟
- tag/release/CI نسخه/ریلیز چطور انجام می‌شه؟

این ۸ سوال رو روز اول بپرس – سطح آمادگی‌ت رو بالاتر از انتظارشون نشون می‌ده و استرس «نمیدونم انتظارات چیه» رو می‌کشه.

✓ جمع‌بندی فصل

- **GitHub Flow:** main همیشه deploy-پذیر؛ همه‌چیز از PR رد می‌شه – پیش‌فرض مدرن
- **Git Flow:** main+develop + feature/release/hotfix – برای نسخه‌های موازی
- **Trunk-Based:** شاخه کوتاه + Feature Flags – سبک شرکت‌های بزرگ
- deploy-پذیری main، ریویو و CI، استراتژی merge = تصمیم‌های کلیدی هر workflow
- چک‌لیست ۸ سوال روز اول = واکسن استرس

📝 تمرین فصل ۱۲

1. برای یکی از پروژه‌های خود یک فایل راهنما تدوین کرده و **بنویسید** (فایل `CONTRIBUTING.md`): کدوم workflow، قالب شاخه، قالب کامیت، استراتژی merge. نوشتنش خودش نصف بلوغه!
2. شبیه‌سازی تیمی: دو شاخه فیچر همزمان بساز که هر دو فایل مشترکی رو تغییر بدن؛ اولی رو merge کن، دومی رو با اولی sync کن (`rebase`) و تعارض رو حل کن – دقیقاً همون حس کار تیمی.
3. یکی از مخزن‌های معروف گیت‌هاب رو باز کن (مثلاً `nextjs`) → تب PR ۳ → Pull Requests تازه رو ببین: workflow شون چیه؟ ریویوها چطور نوشته شدن؟ (بخونی روحیه تیم‌های واقعی.)

▼ راهنمای تمرین ۱ (نمونه `CONTRIBUTING.md`)

```
# Contributing
- Workflow: GitHub Flow – main همیشه deploy-پذیر
- Branches: `feat/<scope>` | `fix/<scope>` | `docs/<scope>`
- Commits: Conventional Commits (feat/fix/chore/docs/...)
- PRs: approval کوچک، تک‌موضوع، با توضیح «چه/چرا/تست»، حداقل ۱
- Merge: squash and merge
```

➔ **فصل بعد:** کتاب نجات – ۱۵ سناریوی «اوپس» با راه‌حل دقیق. کتابی که می‌خواهی همیشه کنارت باشه.

فصل ۱۳ – کتاب نجات: «چه کار کنم اگر...» 🚚

🎯 هدف: ۱۵ سناریوی واقعی که دولوپرها رو در عرق می‌بره – با راه‌حل دقیق و ایمن. این فصل رو bookmark کن؛ روز حادثه، نسخه‌ی PDF رو باز کن، سناریو رو پیدا کن، اجرا کن، استرس صفر.

⚠️ سه قانون کلی هر حادثه: ۱. دستور عجولانه نزن – بیشتر فاجعه‌ها با «برای حل، سریع چند چیز دیگه زدم» ساخته می‌شن ۲. اول `git status` و `git lg --all` و (در موارد جدی) `git reflog` – وضعیت واقعی رو ببین ۳. کد کامیت‌شده تقریباً هیچ‌وقت از بین نمی‌ره (۹۰ روز حداقل!) – بدترین حالت‌ها برگشت‌پذیرند

۱. «کامیت رو روی شاخه اشتباه زدم!» 🚚

تازه کامیت زدی و دیدی روی `main` هستی، نه `feature/x`:

```
# نکردی - کامیت‌ها رو با خودت ببر push هنوز
git switch feature/x
git cherry-pick <SHA> # کامیت به شاخه درست رفت
git switch main
git reset --hard HEAD~1 # بردارش main از
```

یا با `git reset --soft HEAD~1` روی `main` و بعد `switch` و کامیت مجدد. کلید: `soft` = کد نگه داشته می‌شه.

۲. «کامیت زدم ولی یک فایل رو جا انداختم» 🚚

```
git add forgotten-file.js
git commit --amend --no-edit # نشده؟ عالی، بی‌سروصدا کاملش کن push هنوز
```

اگه `push` شده: یک کامیت جدید بزن (`amend`) – `git commit -m "fix: add missed file"` روی شاخه مشترک ممنوع.

۳. «پیام کامیت اشتباهه» 🚚

همون سناریوی بالا: `amend --` قبل از `push`؛ بعد از `push` روی شاخه شخصی: `push --force-with-lease` + `amend`؛ روی شاخه مشترک: می‌سوزه، بذار بمونه 😊 (یا در PR از `squash` استفاده کن که پیام نهایی رو تو می‌نویسی).

۴. «وسط کارم، main تغییر کرده و pull نمی‌خوام merge commit بگیرم» 🚚

```
git stash -u          # کامیت موقت بزن
git pull --rebase      # تاریخچه خطی
git stash pop
```

۵. «push رد شد: (fetch first) rejected» 🚚

یعنی ریموت جلوتره. هیچ force نزن!

```
git pull --rebase origin main      # کار ریموت رو بگیر، کامیتهات روش بشین
# تعارض؟ add + rebase --continue
git push
```

۶. «rebase/amend زدم و حالا push نمی‌شه – یعنی همه‌چیز خراب شد؟» 🚚

نه – فقط SHA ها عوض شدن. اگر قبلاً push شده بود:

```
git push --force-with-lease      # فقط روی شاخه‌ی خودت!
```

اگر شک داری که چیزی گم شده: `git reflog` – هر حالت قبلی اونجاست (فصل ۶).

۷. «توی Detached HEAD هستم!» 🚚

`git status` می‌گه: `HEAD detached at a1b2c3d` – یعنی مستقیم روی یک کامیت هستی نه شاخه. خطرناک نیست؛

فقط کامیت‌های جدید اینجا «بی‌خانه» می‌شن:

```
# کاری که اینجا کردی رو می‌خوای نگه داری:
git switch -c new-branch      # از همین‌جا شاخه بساز

# فقط می‌خوای برگردی:
git switch main               # (یا هر شاخه‌ای)
```

۸. «Merge conflict گرفتم و گیج شدم – راه خروج؟» 🚚

```
git merge --abort          # merge: به قبل از
git rebase --abort         # rebase: به قبل از
# ... حالا با ذهن آروم دوباره، یا از یک همکار کمک بگیر ...
```

عقب‌نشینی همیشه مجازه؛ هیچ‌کس به خاطر abort کردن قضاوت نمی‌کنه – به خاطر push کردن کد نصفه قضاوت می‌کنن!

۹. «فایل حساس (کلید/پسورد/.env) در کامیت رفته!» 🚚

دو مرحله – به این ترتیب:

- ۱) اول امنیت، بعد گیت: کلید لو رفته = لو رفته. هرگز فکر نکن «حذفش می‌کنم کافی است» – بات‌های گیت‌هاب ظرف دقیقه‌ها مخزن‌های عمومی رو اسکن می‌کنن. → کلید/توکن رو فوراً rotate کن (باطلش کن و جدید بساز) و از gitscanning ایمیلی که گیت‌هاب فرستاده (Secret scanning alert) پیگیری کن.
- ۲) بعد پاکسازی تاریخچه: فایل در تمام تاریخچه‌هاست؛ یک commit حذف ساده کافی نیست:

```
git filter-repo --path .env --invert-paths # ابزار استاندارد (pip install git-filter-repo)
# BFG Repo-Cleaner یا برای فایل‌های متنی: جایگزینی رشته در کل تاریخچه با
git push --force-with-lease
```

و برای آینده: .env رو از روز اول gitignore کن (فصل ۹) و .env.example بذار.

۱۰. «چند کامیت زدم که کلاً باید نبودن» 🚚

```
# نشده push کامیت‌های آخر، هنوز:
git reset --hard HEAD~2
git reset --soft HEAD~2
# دوتای آخر کلاً حذف (کد هم می‌ره!)
# کد می‌مونه، کامیت‌ها نه – بعد کامیت تمیز جدید
```

push شده؟ فقط revert (فصل ۶).

۱۱. «شاخه‌ای رو حذف کردم که نباید حذف می‌شد» 🚚

```
git reflog
git switch -c restored <SHA-قدیم> # با: git log --oneline --reflog
```

شاخه = اشاره‌گر بود؛ اشاره‌گر بردی، کامیت‌ها سرچاشون‌ان (فصل ۷، تمرین ۲).

۱۲. repository همکارم به پروژه‌م وصل می‌شه ولی diff ها عجیبین – خط آخر هر

فایل!» 🚚

کلاسیک CRLF (فصل ۲): تیم مختلط ویندوز/مک. علاج: gitattributes. با * text=auto eol=lf در مخزن + یک بار re-normalize:

```
git add --renormalize .
git commit -m "chore: normalize line endings"
```

۱۳. «می‌خواهم فقط یک فایل خاص رو از کامیت دیگه بگیرم» 🚚

```
git checkout a1b2c3d -- src/config.js      # نسخه‌ی آن کامیت از این فایل
# (دستور جدید: git restore --source=a1b2c3d src/config.js)
git commit -m "chore: align config with v1.4"
```

۱۴. «commit زدم با ایمیل شخصی روی پروژه شرکت!» 🚚

```
# ایمیل شرکتی را تنظیم کن:
git config user.email "developer@company.com"  # فقط همین مخزن: global: --بدون
# نشده؟ push آخرین کامیت هنوز
git commit --amend --reset-author --no-edit
```

نکته: کمیت‌های گیت‌هاب با ایمیل متصل به اکانتت به پروفایلت لینک می‌شن – برای شرکت معمولاً ایمیل سازمانی جدا می‌خوان.

۱۵. «باگه ولی نمی‌دونم کدوم کامیت معرفی‌ش کرد» 🚚

```
git bisect start
git bisect bad          # وضعیت فعلی خرابه
git bisect good v1.4.0  # این نسخه سالم بود
# تا مقصر پیدا شود - good/bad گیت نصفه می‌رود؛ تست کن و بگو
git bisect reset
```

(فصل ۹ کاملش رو دیدی.)

🧰 جعبه ابزار نجات – خلاصه یک‌نگاهی

ابزار	نقش در حوادث
<code>git status</code> / <code>git lg --all</code>	اولین قدم همیشه: ببین واقعاً کجایی
<code>git reflog</code>	نجات هر کامیت «گم‌شده»
<code>git revert</code>	خنثی‌سازی امن روی تاریخچه مشترک
<code>git reset --soft</code>	برداشتن کامیت‌ها با نگه‌داشتن کد
<code>git stash</code>	محافظت از کار نیمه‌تمام موقع مانور
<code>merge/rebase --abort</code>	عقب‌نشینی از عملیات نیمه‌تمام
<code>--force-with-lease</code>	تنها <code>force</code> پذیرفتنی – و فقط شاخه شخصی
<code>filter-repo</code> / BFG	پاکسازی تاریخچه (بعد از <code>rotate</code> کردن رازها!)

👉 حرف آخر استادی: هیچ‌کدوم از این سناریوها با خوندن حادثه نمی‌شن – با تمرینِ حادثه می‌شن. زمین تمرین رو ببر توی این وضعیت‌ها (عمداً!) و نجاتشون بده. روزی که بتونی با لبخند بگی «اوپس، `reflog` می‌زنم»، در شرکت استرست صفره.

📝 تمرین فصل ۱۳ (آزمون آتش‌نشانی 🔥)

هر ۵ سناریوی زیر رو عمداً در زمین تمرین بساز و نجات بده:

1. کامیت روی شاخه اشتباه → جابجایی با `cherry-pick + reset`
2. `reset --hard` روی ۲ کامیت → نجات با `reflog`
3. شاخه رو با `-D` حذف کن → بازسازی از `reflog`
4. `push` به شاخه‌ای که ریموت جلوتره → `pull --rebase` و `push` موفق
5. Detached HEAD با `git switch <SHA>` → ساخت شاخه از آنجا و برگشت امن

▼ چک‌لیست جواب‌ها (خلاصه)

```
# 1) cherry-pick <sha> روی شاخه درست + reset --hard HEAD~1
# 2) git reflog → git reset --hard HEAD@{n}
# 3) git reflog → git switch -c restored <sha>
# 4) git pull --rebase && git push
# 5) git switch -c keep-this && git switch main
```

➡ **فصل بعد:** GitHub Actions و ابزارهایی که حرفه‌ای‌ها دور گیت می‌چینن.

فصل ۱۴ – Git Hooks: نگهبان‌های خودکار روی خط تولید

🎯 هدف: اسکریپت‌هایی که گیت خودش در لحظه‌های مشخص اجرا می‌کند – قبل از کامیت، روی پیام، قبل از push. نتیجه: خطاهای تکراری (پیام بی‌قاعده، .env، توی کامیت، push با تست قرمز) از «یادم نمی‌آد» تبدیل می‌شه به «غیرممکنه».

۱۴.۱ – Hook چیست؟

Hook یعنی «قلاب»: گیت در نقاط مشخص چرخه‌اش می‌پرسه «اسکریپتی داری اجرا کنم؟». دو خانواده:

خانواده	کجا اجرا می‌شه	مثال	سرنوشت در دنیای واقعی
Client-side	روی سیستم تو	pre-commit ، commit-msg ، pre-push ، post-merge	درسی همین فصل – با Husky به تیم می‌رسه
Server-side	روی سرور مخزن (قبل از قبول push)	pre-receive ، update	روی گیت‌هاب بهش دسترسی نداری – معادلش Rulesets و CI است (فصل ۱۱ و ۱۵)

🎯 قاعده طلایی هوک‌ها: exit code همه‌چیز است. اسکریپت با ۰ تمام شود = «اجازه بده»؛ با غیر صفر = «عملیات لغو». به همین سادگی یک نگهبان می‌سازی.

هوک‌های کلید که در این فصل می‌سازیم:

- pre-commit – قبل از ساخت کامیت (محتوا را ببین، اجازه/رد)
- commit-msg – پیام کامیت را ببین (محتوای کامیت را نه!)
- pre-push – آخرین سد قبل از رفتن به سرور
- post-merge – بعد از pull/merge موفق (مثلاً npm install خودکار)

۱۴.۲ – تور پوشه hooks و فعال‌سازی

یادته در تور .git (فصل ۴) پوشه‌ای دیدی که ورش نداشتیم؟ حالا نوبتشه:

```
ls .git/hooks/
# applypatch-msg.sample  pre-commit.sample  commit-msg.sample ...
# pre-push.sample        post-merge.sample  ...
```

- هر `.sample` یک نمونه‌ی آماده و غیرفعال است – سند رسمی: فقط کپی‌ش کن بدون پسوند
- اسم فایل دقیقاً باید نام هوک باشد (`pre-commit`)، بدون پسوند
- در Git Bash باید executable بشه: `chmod +x .git/hooks/pre-commit`
- زبان‌ش فرقی نمی‌کند: شل، پایتون، نود... هر برنامه‌ی قابل اجرا با shebang درست

```
cp .git/hooks/pre-commit.sample .git/hooks/pre-commit
chmod +x .git/hooks/pre-commit # فعال شد - همین!
```

🔔 نکته تیمی مهم: چون هوک‌ها داخل `.git` اند، نسخه‌بندی نمی‌شن – با `push` به همکاری نمی‌رسن! راه‌حل: بخش ۱۴.۶ (`core.hooksPath` و Husky).

۱۴.۳ – اولین نگهبان: `pre-commit`

سناریوی واقعی: هیچ‌وقت نباید `.env` (یا هر فایل حساس/زباله‌ی مشخص) کامیت بشه – حتی اگر یک بار وسط شب حواست پرت شد:

```
#!/bin/sh
# .git/hooks/pre-commit - اجازه نمی‌دهد .env کامیت شود
if git diff --cached --name-only | grep -q "\.env$"; then
    echo "❌ .env نمی‌تواند کامیت شود"
    echo "اصلاح: git restore --staged .env"
    exit 1
fi
exit 0
```

ابزاری که اینجا نجات می‌دهد:

```
git diff --cached --name-only # در این کامیت staged فهرست دقیق فایل‌های
```

تستش کن: `.env` را بساز، `git add` کن و کامیت بزن – گیت عملیات را رد می‌کند و پیام تو را می‌بینی. کامیت رخ نمی‌دهد، کدهای `staged` هم دست‌نخورده می‌مانند (فقط اجازه داده نشد).

💡 ایده‌های `pre-commit` واقعی: بازداشتن `console.log` در شاخه‌های `release`، جلوگیری از کامیت فایل‌های `*.local`، اجرای `eslint` روی فایل‌های `staged`. (فایل حساس رفته در تاریخچه؟ دیر شد – فصل ۱۳ سناریوی ۹.)

۱۴.۴ – `commit-msg`: پلیس `Conventional Commits` (پیوند فصل ۳)

فصل ۳ استاندارد پیام‌ها (`fix:`، `feat:` و...) رو یاد گرفتی. حالا تضمینش کن:

```
#!/bin/sh
# .git/hooks/commit-msg - باید پیام Conventional باشد
msg="$(cat "$1")"
pattern="^(feat|fix|chore|docs|style|refactor|perf|test|build|ci)(\(.+\))?!?: .+"
if ! echo "$msg" | grep -qE "$pattern"; then
  echo "❌ مثل: feat: add login form"
  echo "کامیت لغو شد - پیام را اصلاح و دوباره کامیت بزن"
  exit 1
fi
```

دو نکته ظریف: - آرگومان \$1 مسیر یک فایل موقت است که پیام داخلش - هوک پیام را می‌بیند، نه محتوای کامیت را - این هوک بعد از ساخت پیام اجرا می‌شود؛ رد شدن یعنی کامیت اصلاً ساخته نمی‌شود

این دقیقاً همون چیزیه که در پروژه‌های جدی می‌بینی: پیام‌های ناقص قلقلان، هیچ‌وقت وارد تاریخچه نمی‌شن.

۱۴.۵ - pre-push: آخرین سد قبل از سرور

تست‌ها رو قبل از هر push اجرا کن - تا «رک گول زدن CI» اتفاق نیفته (فصل ۱۵):

```
#!/bin/sh
# .git/hooks/pre-push - ممنوع push، بدون تست سبز
npm test --silent || { echo "❌ تست‌ها رد شدند"; exit 1; }
```

و یک کاربرد دوستانه‌تر از خانوادگی post:

```
# .git/hooks/post-merge - وابستگی‌ها تازه باشند: pull/merge بعد از هر
npm install --silent
```

۱۴.۶ - هوک در تیم: Husky و core.hooksPath

چون هوک‌های .git/hooks با مخزن سفر نمی‌کنند، برای تیم دو راه داریم:

راه ۱ - خود گیت: پوشه‌ای نسخه‌بندی‌شده بساز و گیت را به آن اشاره بده:

```
mkdir .githubhooks
# بگذار و کامیت کن؛ سپس هر عضو تیم یک بار .githubhooks/ هوک‌ها را داخل:
git config core.hooksPath .githubhooks
```

راه ۲ - استاندارد پروژه‌های JS: Husky + lint-staged:

```
npm install --save-dev husky lint-staged
npx husky init # تنظیم می‌شود + core.hooksPath ساخته می‌شود
echo "npx lint-staged" > .husky/pre-commit
```

```
// package.json - کن auto-fix را فقط staged فایل‌های
{
  "lint-staged": {
    "*.{js,ts,tsx}": [ "eslint --fix", "prettier --write" ]
  }
}
```

از این به بعد هر کامیت، فقط فایل‌های درگیر را lint و format می‌کند - همون جادویی که وقتی در شرکت کامیت می‌زنی و «auto-fix» می‌شه (فصل ۱۵، جدول ابزارها). اینه عاملش!

۱۴.۷ - پرش از هوک: --no-verify و اخلاق کار

هر هوک client-side قابل رد شدن است:

```
git commit --no-verify -m "wip" # نمی‌خواند commit-msg و pre-commit هوک‌های
git push --no-verify # را رد می‌زند pre-push
```

- یک بار در شرایط اضطرار؟ طبیعیه. عادت کردن بهش؟ یعنی هوکت یا بد نوشته شده یا واقعاً لازم نیست - درستش کن، نه این‌که هر روز پرش بزنی
- چون client-side قابل رد شدن و غیرقابل اعتماد برای اعمال اجباری است، چک‌های قطعی همیشه **چندلایه‌اند**: هوک محلی (سرعت) + CI روی گیت‌هاب (فصل ۱۵) + protected branch (فصل ۱۱) - دفاع در عمق، نه یک نگهبان تنها

✓ جمع‌بندی فصل

- هوک = اسکریپت در نقطه‌ای از چرخه گیت؛ exit 0 = ادامه، غیرصفر = لغو
- `pre-commit` محتوا را چک می‌کند (`commit-msg`)، (`git diff --cached`) پیام را (`$1` = فایل پیام)، `pre-push` آخرین سد است
- هوک‌ها در `.git/hooks` اند و **نسخه‌بندی نمی‌شوند** → برای تیم: `core.hooksPath` یا Husky + lint-staged
- `--no-verify` پرش است، نه راه حل؛ اعمال اجباری همیشه در CI و Rulesets تکرار می‌شود

📝 تمرین فصل ۱۴ (همه در زمین تمرین)

1. `pre-commit` بخش ۱۴.۳ را بساز، `chmod +x` بزن، و با یک `staged .env` شده تست کن - خروجی چیست؟ بعد `--no-verify` را امتحان کن.
2. `commit-msg` بخش ۱۴.۴ را نصب کن و با "چیزی که گذاشتم" `git commit -m` ردش کن؛ بعد با پیام معتبر کامیت موفق بزن.
3. یک `pre-push` بنویس که `git status --porcelain` چک کند - اگر فایل `untracked` مونده، `push` رد شود.
4. (چالش) `post-merge` بنویس که تاریخ آخرین `pull` را در یک فایل `.last-pull` لاگ کند؛ `pull` بزن و فایل را ببین.

▼ جواب تمرین ۳

```
#!/bin/sh
# .git/hooks/pre-push - push فقط با working directory مرتب
if [ -n "$(git status --porcelain)" ]; then
    echo "🚫 working directory کثیف است (untracked/modified) - اول add/commit یا stash"
    exit 1
fi
```

`git status --porcelain` خروجی قابل اسکرپت `status` است: خالی = تمیز.

➔ **فصل بعد:** اتوماسیون سمت گیت‌هاب – GitHub Actions و ابزارهای حرفه‌ای دور گیت؛ جایی که نگهبان‌ها از

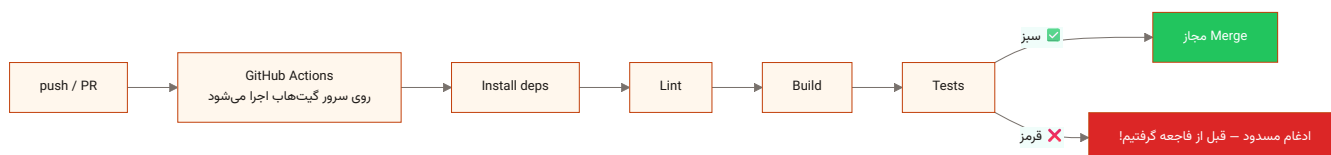
سیستم تو به سرور مهاجرت می‌کنند.

فصل ۱۵ – GitHub Actions و ابزارهای حرفه‌ای دور گیت

🎯 هدف: یاد بگیری CI (یکپارچه‌سازی پیوسته) چیه و یک workflow واقعی روی مخزن خودت راه بندازی، با ابزارهایی که پروژه‌های حرفه‌ای رو «حرفه‌ای» می‌کنن: Checks سبز، Dependabot، Badges و امضای کامیت.

۱۵.۱ – CI چیست و چرا در همه شرکتی هست؟

CI (Continuous Integration) یعنی: با هر push و هر PR، ماشین‌ها به جای انسان چک‌های خودکار را اجرا کنند – lint، build، تست‌ها. نتیجه: کد خراب قبل از رسیدن به main گرفته می‌شه، نه بعد از deploy در ساعت ۲ بامداد.



در PR فصل ۱۱ دیدی: **Checks** همون خروجی CI است. در شرکت‌ها merge تا سبز شدن checks قفل می‌مونه.

۱۵.۲ – اولین Workflow روی مخزن خودت

GitHub Actions = موتور اجرای خودکار گیت‌هاب. دستور کارها در فایل‌های **YAML** داخل پوشه `.github/workflows/` مخزن نوشته می‌شن.

یک workflow واقعی برای یک پروژه Next.js/Node بساز – فایل `.github/workflows/ci.yml`:

```

name: CI

on:
  push:
    branches: [main]
  pull_request:      # روی همه PRها

jobs:
  test:
    runs-on: ubuntu-latest      # ماشین مجازی تمیز برای هر اجرا
    steps:
      - uses: actions/checkout@v4      # کن clone اکشن آماده: کد مخزن را

      - uses: actions/setup-node@v4
        with:
          node-version: 22
          cache: npm

      - run: npm ci                # package-lock نصب تمیز و دقیق طبق
      - run: npm run lint
      - run: npm run build
      - run: npm test

```

خطبه خط بفهم:

معنی	بخش
اسم workflow (نمایش در تب Actions)	name: CI
رویدادهای ماشه	on: push / pull_request
هر job روی یک ماشین مجازی تازه اجرا می‌شه (موازی)	jobs
ترتیب کارها؛ uses: = اکشن آماده دیگران، run: = دستور شل	steps
بدون این، ماشین هیچ کدی نداره! اولین step تقریباً همیشه همیشه	actions/checkout@v4

بعد:

```

mkdir -p .github/workflows
# فایل را بساز و:
git add .github && git commit -m "ci: add lint, build and test workflow"
git push

```

برو به مخزن → تب **Actions** — اجرای زنده رو ببین: هر step با لاگ کاملش. بعد از این، در PR ها بخش **Checks** سبز/قرمز می‌شه.

💡 نکته فرانت‌کار: این دانش مستقیماً قابل فروش است. «CI راه می‌ندازم» جمله‌ای است که در مصاحبه وزن داره و در عمل، ۹۰٪ پروژه‌ها فقط به همین ۱۲ خط یامل نیاز دارن.

۱۵.۳ – Badges: نشان‌های وضعیت روی README

آن آیکون‌های کوچیک بالای README پروژه‌ها (build status، نسخه، لایسنس):

```
! [CI](https://github.com/USER/REPO/actions/workflows/ci.yml/badge.svg)
```

از صفحه Actions روی workflow خودت، دکمه «...» → **Create status badge** – کد آماده‌ش رو کپی کن بالای README. نشان سبز = «این پروژه نگهداری می‌شه» به چشم هر بازدیدکننده‌ای (و مصاحبه‌گری).

۱۵.۴ – Dependabot: نگهبان وابستگی‌ها

دیپندنسی‌های قدیمی = حفره امنیتی. Dependabot (داخل خود گیت‌هاب) به‌صورت دوره‌ای PR های آپدیت خودکار باز می‌کنه:

فعال‌سازی با فایل `.github/dependabot.yml`:

```
version: 2
updates:
  - package-ecosystem: npm      # برای npm / pnpm: npm
    directory: "/"
    schedule:
      interval: weekly
  - package-ecosystem: github-actions
    directory: "/"
    schedule:
      interval: weekly
```

بعدش هفته‌ای چند PR می‌بینی با عنوان `Bump next from 15.x to 15.y` – صرفاً merge کن (checks سبزه) یا نگاهی بنداز. در شرکت هم معمولاً Dependabot یا ابزار مشابه (Renovate) روشنه – پس چیز غریبه‌ای نیست.

۱۵.۵ – امنیت کامیت: امضای دیجیتالی (Signed Commits)

چرا کنار بعضی کامیت‌ها علامت «Verified» هست؟ چون با کلید GPG/SSH امضا شدن – یعنی واقعاً از سیستم صاحب اکانت اومدن، نه کسی که اسم و ایمیلش رو جعل کرده (چون تنظیم `user.name/email` برای هرکس ممکنه!).

فعال‌سازی با SSH (ساده‌ترین راه مدرن):

```
# فصل ۲ برای امضا استفاده کن SSH از همون کلید:
git config --global gpg.format ssh
git config --global user.signingkey ~/.ssh/id_ed25519.pub
git config --global commit.gpgsign true

# کلید عمومی را در گیت‌هاب اضافه کن:
# Settings → SSH and GPG keys → New SSH key → Key type: Signing Key
```

از این به بعد کامیت‌ها روی گیت‌هاب **Verified** می‌شن – جزئیات کوچیکی که پروفایل حرفه‌ای رو می‌سازه (و در شرکت‌های امنیت‌محور گاهی الزامی است).

۱۵.۶ – ابزارهای مکملی که تو اکوسیستم می‌بینی (شناختی، نه تسلط)

ابزار	چیست	چرا اسمش را می‌شنوی
GitLens	افزونه VS Code: تاریخچه/blame فوق حرفه‌ای داخل ادیتور	نصبش کن – مال روز اول است
Husky + lint-staged	گیت هوک‌ها در پروژه‌های JS: قبل از هر کامیت، lint/format خودکار	در اکثر پروژه‌های شرکتی نصبه – وقتی کامیت می‌زنی و «auto-fix» اجرا می‌شه، اینه
git-flow / gh-cli	افزونه شاخه‌بندی / CLI رسمی گیت‌هاب (gh pr create و ...)	gh رو نصب کن؛ ساخت PR از ترمینال رایجه
CodeSandbox/Codespaces	VS Code ابری متصل به مخزن	دکمه سبز «Code → Codespaces» در مخزن‌ها
Secret scanning / push protection	گیت‌هاب جلوی push کردن کلید لو رفته رو می‌گیره	اگه روزی push رد شد با پیام secret، خونه‌ای – داره نجات می‌ده

نمونه gh – ساخت PR بدون باز کردن مرورگر:

```
gh auth login          # یک بار
gh pr create --fill    # عنوان/توضیح از کامیت‌ها
gh pr checks           # وضعیت checks
gh pr merge --squash --delete-branch # ادغام + حذف شاخه، همه از ترمینال
```

۱۵.۷ – نگهداری حرفه‌ای مخزن (چک‌لیست پروژه بالغ)

- [] README با بخش‌های: چی هست / نصب / اجرا / ساختار پوشه‌ها
- [] Badge های CI و لایسنس
- [] .env.example + .gitignore (بدون مقدار واقعی)
- [] CI workflow سبز روی هر PR
- [] Dependabot روشن
- [] Releases با تگ semver + release notes (فصل ۹)
- [] قالب PR و ایشو (.github/pull_request_template.md و ISSUE_TEMPLATE)
- [] امضای کامیت‌ها (Verified)

این چک‌لیست استانداردهای مهندسی را روی پروژه‌های خود پیاده‌سازی کنید تا مخزن‌ها از وضعیت صرفاً «کد ذخیره‌شده» به یک «محصول مهندسی‌شده و استاندارد» ارتقا پیدا کنند؛ این همان الگویی است که تیم‌های حرفه‌ای در بررسی‌های فنی انتظار دارند.

جمع‌بندی فصل ✓

- CI = چک‌های خودکار روی هر push/PR؛ در گیت‌هاب با **GitHub Actions** و **YAML** داخل `.github/workflows/`
- ساختار حداقلی: **on** (ماشه) + **uses** (**steps** + **jobs** اکشن آماده، **run** شل)؛ `actions/checkout` تقریباً همیشه اولین **step**
- **Checks** سبز = شرط **merge** در شرکت‌ها؛ **Badge** = تبلیغ زنده سلامت پروژه
- **Dependabot** آپدیت وابستگی‌ها را **PR** خودکار می‌کند؛ **push protection** جلوی لو رفتن رازها را می‌گیرد
- **CLI** **gh** و **GitLens** و **Husky** = ابزارهای روزمره‌ای که در شرکت‌ها همه‌جا هستند
- **Signed commits** = **Verified** روی کارت ویزیت حرفه‌ای‌ات

تمرین فصل ۱۵ 📄

1. روی یکی از پروژه‌های خود، فایل `ci.yml` این فصل را پیاده‌سازی و **push** کنید. تب **Actions** را ببین — سبز شد؟ (اگه تست نداری، فقط **lint** و **build** نگه دار).
2. **Badge** ساخته‌شده را بالای **README** بگذار.
3. `dependabot.yml` را اضافه کن و یک هفته بعد به **PR** های بازشده سر بزن.
4. **CLI** **gh** را نصب و `gh auth login` کن؛ بعد یک **PR** را کاملاً از ترمینال بساز و **merge** کن.
5. (اختیاری ★) امضای **SSH** کامیت‌ها را فعال کن و **Verified** را ببین.

❗ عیب‌یابی سریع CI

- «nothing to run»: مسیر فایل حتماً `.github/workflows/*.yml` باشد
- قرمز روی `package-lock.json` : `npm ci` را **commit** کرده‌ای؟ (اگر **npm** استفاده می‌کنی، **lock file** باید در مخزن باشد)
- قرمز روی **build**: اول خودت لوکال `npm run build` بزن — **CI** فقط همون را تکرار می‌کند
- لاگ هر **step** را باز کن؛ پیام خطا تقریباً همیشه خودگویاست

➡ **فصل آخر:** چیت‌شیت‌ها، گلاساری و ۲۰ سوال مصاحبه با جواب کوتاه.

فصل ۱۶ – چیت‌شیت، گلاساری و ۲۴ سوال مصاحبه

🎯 هدف: مرجع مرور سریع – روی دیوار اتاق، کنار مانیتور، و شب قبل از مصاحبه.

```

# === شروع ===
git init                # مخزن جدید
git clone <url>         # کپی مخزن ریموت
git status / git st    # خلاصه (sb - دستور) وضعیت

# === ثبت ===
git add file / git add -p    # (p - تکه‌تکه با) به سبد
git mv old new              # جابه‌جایی/تغییر نام زیر نظر گیت
git rm file / git rm --cached f    # حذف (از دیسک+مخزن / فقط مخزن)
git commit -m "msg"          # ثبت
git commit --amend           # (push! فقط قبل از) اصلاح آخرین
git restore file             # برگشت ویرایش‌ها (⚠ پاک می‌کند)
git restore --staged file    # فقط unstage

# === دیدن ===
git lg                    # گراف همه شاخه‌ها (alias)
git diff / diff --staged / diff HEAD    # جزئیات یک کامیت
git show <sha>            #
git log --author="Ali" --since="2 weeks ago"    # pickaxe: شکار رشته در تغییرات
git log -S "apiKey" --oneline    # کامیت‌هایی که شاخه ویژگی به شاخه اصلی اضافه می‌کند
git log main..feature/x        # هر خط، کدوم کامیت
git blame -L 10,20 -- src/app.js    # آمار کامیت هر نفر
git shortlog -sn              #

# === شاخه ===
git switch -c feature/x    # ساخت + پرش
git switch -                # شاخه قبلی
git merge feature/x        # (add → تعارض: حل) ادغام
git branch -d / -D name    # حذف امن / اجباری

# === تاریخچه تمیز ===
git rebase main             # main انتقال کامیت‌های شاخه جاری روی نوک
git rebase -i HEAD~4        # جراحی: squash/fixup/reword/drop
git cherry-pick <sha>       # نقل یک کامیت
# ⚠ revert → بازنویسی فقط روی شاخه شخصی؛ مشترک

# === ریموت ===
git fetch                  # فقط دانلود (امن)
git pull --rebase          # دانلود + خطی ادغام
git push -u origin branch  # جفت‌سازی + push اولین
git push --force-with-lease # فقط شاخه شخصی!
git remote -v / add / set-url

# === نجات ===
git reflog                 # نجات هر چیز - HEAD ژورنال
git reset --soft/mixed/--hard HEAD~1    # خنثی‌سازی امن (مشترک)
git revert <sha>           #
git stash / pop / list / apply
git merge --abort / rebase --abort
git clean -n / -fd         # dry-run / حذف untracked
git bisect start            # شکار کامیت معرفی‌کننده باگ

# === زیر پوست ===
git cat-file -t / -p <sha>  # نوع / محتوای یک آبجکت
git ls-tree HEAD [dir]     # (پوشه) tree محتوای یک
git rev-parse main         # کامل شاخه SHA
git count-objects -vH      # آمار و حجم مخزن
git gc                    # فشرده‌سازی و پاکسازی

# === هوک‌ها (فصل ۱۴) ===
ls .git/hooks/             # (sample) نمونه‌های فعال‌نشده
chmod +x .git/hooks/pre-commit    # Git Bash فعال‌سازی در
git commit --no-verify     # پرش از هوک (فقط در اضطرار!)
npx husky init             # (JS) هوک نسخه‌بندی‌شده برای تیم

```

```
# == نسخه ==  
git tag -a v1.2.0 -m "msg" && git push origin v1.2.0
```

۱۶.۲ – چیت‌شیت Conventional Commits

```
feat: فیچر جدید          → MINOR (0.x.0)  
fix: رفع باگ              → PATCH (0.0.x)  
refactor / perf / test / docs / chore / style  
BREAKING CHANGE یا !      → MAJOR (x.0.0)  
  
feat(payments): add refund webhook  
fix(auth): prevent double submit on slow network  
refactor(api)!: change response shape    ← ! = breaking  
  
«چه / چرا / چطور تست شد» PR قالب
```

۱۶.۳ – گلاساری فارسی-انگلیسی

انگلیسی	فارسی	یک جمله
Repository (Repo)	مخزن	پوشه‌ای که تاریخچه گیت دارد
Commit	کامیت / ثبت	snapshot با شناسه و پیام
Staging Area	ناحیه آماده‌سازی	سید خرید کامیت بعدی
Branch	شاخه	اشاره‌گر متحرک به یک کامیت
Merge	ادغام	ترکیب دو خط تاریخچه
Conflict	تعارض	دو تغییر ناسازگار روی یک خط – سوال، نه ارور
Rebase	بازپایه‌گذاری	کندن کامیت‌ها و بکار دوباره روی نوک شاخه دیگر
Remote	ریموت / مخزن دور	آدرس نسخه ابری (origin)
Push / Pull / Fetch	ارسال / کشیدن / دریافت	ارسال کامیت‌ها / دانلود+ادغام / فقط دانلود
Fork	انشعاب	کپی مخزن دیگران در اکانت خودت
Pull Request (PR)	درخواست ادغام	گفتگوی بازبینی قبل از merge
Code Review	بازبینی کد	چشم دوم روی تغییرات
HEAD	–	«الان کجای تاریخچه‌ام»
Detached HEAD	جدا HEAD	ایستادن مستقیم روی کامیت، نه شاخه
Upstream	بالادست	مخزن اصلی (در رابطه فورک)

انگلیسی	فارسی	یک جمله
Tag / Release	برچسب / انتشار	نشانه نسخه رسمی
Stash	ذخیره موقت	کمد موقت تغییرات نیمه‌تمام
Reflog	—	ژورنال حرکات HEAD – بیمه زندگی
Squash	له کردن	ادغام چند کامیت در یک کامیت تمیز
CI / Pipeline	یکپارچه‌سازی پیوسته	چک‌های خودکار روی هر push/PR
Protected branch / Ruleset	شاخه محافظت‌شده	main فقط از دروازه PR
Blob / Tree	—	آبجکت محتوای فایل / آبجکت پوشه در دیتابیس <code>.git</code>
Packfile	بسته	فشرده‌سازی مجموعه آبجکت‌ها (خروجی <code>git gc</code>)
Hook	قلاب	اسکرپت خودکار در نقاط چرخه گیت (pre-commit و...)
Issue	ایشو / کار	برگه باگ/فیچر/سوال – <code>Fixes #142</code> بعد از merge می‌بنددش
Pickaxe	—	جست‌وجوی یک رشته در تغییرات تاریخچه (<code>git log -S</code>)

۱۶.۴ – ۲۴ سوال رایج مصاحبه (با جواب‌های فشرده)

۱. گیت و گیت‌هاب چه فرقی دارند؟ گیت ابزار کنترل نسخه روی سیستم شماست؛ گیت‌هاب سرویس ابری میزبانی مخزن‌ها با قابلیت‌های همکاری (PR، ایشو، CI). مثل فرق Word و Google Docs.

۲. `commit` چه اطلاعاتی دارد؟ Snapshot کامل پروژه + SHA هش‌شده از محتوا + اشاره به کامیت پدر + نویسنده، تاریخ، پیام.

۳. `working directory`، `staging` و `repository`؟ میز کار → `add` → سبد کامیت → `commit` → انبار دائمی. `add` نشده، کامیت نمی‌شود.

۴. شاخه در گیت واقعاً چیست؟ یک فایل متنی کوچک حاوی SHA یک کامیت – اشاره‌گر متحرک. برای همین ساختنش رایگان و آنی است.

۵. فرق `merge` و `rebase`؟ هر دو دو شاخه را ادغام می‌کنند. `merge` کامیت جدیدی با دو والد می‌سازد (تاریخچه واقعی انشعاب). `rebase` کامیت‌های شاخه را با SHA جدید روی نوک شاخه مبنا بازسازی می‌کند (تاریخچه خطی) – و چون بازنویسی تاریخچه است، فقط روی شاخه‌های محلی/اختصاصی امن است.

۶. `merge conflict` چیست و چطور حل می‌شود؟ دو تغییر ناسازگار روی یک ناحیه از یک فایل. گیت نشان‌های <<<<<< می‌گذارد؛ تصمیم می‌گیرید (حتی با صاحب کد دیگر)، فایل را اصلاح می‌کنید، `add` و `commit` (در >>>>>> =====)

(rebase: `--continue`).

۷. **فرق reset و revert؟** reset شاخه را به عقب بازمی‌گرداند (بازنویسی تاریخچه – مخصوص شاخه‌های محلی). revert کامیت معکوس جدید می‌سازد (تاریخچه دست‌نخورده – امن برای شاخه‌های مشترک).

۸. **git pull و git fetch؟** fetch فقط داندلود به `origin/*` بدون تغییر کد شما؛ `pull = fetch + merge/rebase`. حرفه‌ای: اول fetch و نگاه، بعد ادغام.

۹. **git reset --hard** **زدم و کدم رفت!** `git reflog` بزن؛ SHA حالت قبلی را پیدا و `git reset --hard <SHA>` کن. کامیت‌ها ۹۰~ روز در `git` زنده‌اند.

۱۰. **force push چیست و کی مجاز است؟** بازنویسی ریموت با تاریخچه لوکال. فقط روی شاخه‌های شخصی و فقط با `--force-with-lease` روی `main`/مشترک هرگز.

۱۱. **HEAD چیست؟ Detached HEAD چیست؟** HEAD = اشاره‌گر «موقعیت فعلی» (معمولاً نوک شاخه). اگر مستقیم روی یک کامیت باشد (مثلاً بعد از detached)، `git switch <sha>` است – کامیت جدید بی‌خانه می‌شود؛ راه نجات: `git switch -c` از همان‌جا.

۱۲. **Pull Request چیست و چرا در شرکت‌ها الزامی است؟** درخواست ادغام شاخه به `main` همراه diff، توضیحات، code review و checks خودکار – دروازه کیفیت: هیچ کدی بدون چشم دوم و تست به `main` نمی‌رسد.

۱۳. **squash merge چیست و چرا محبوب است؟** همه کامیت‌های شاخه به یک کامیت تمیز در `main` تبدیل می‌شوند – `main` خوانا می‌ماند و wip ها دور ریخته می‌شوند.

۱۴. **cherry-pick کی لازم می‌شود؟** انتقال یک کامیت مشخص به شاخه دیگر – مثلاً backport فیکس حیاتی از `main` به شاخه release قدیمی.

۱۵. **fast-forward merge چیست؟** اگر از لحظه انشعاب، شاخه مبنا ثابت مانده باشد، merge فقط اشاره‌گر را جلو می‌برد و کامیت merge ساخته نمی‌شود – تاریخچه خطی می‌ماند.

۱۶. **tag و release؟** tag برچسب دائمی روی یک کامیت (نسخه) است – annotated بهتر است؛ release نسخه رسمی گیت‌هایی همان تگ با یادداشت انتشار. `semver: MAJOR.MINOR.PATCH`.

۱۷. **gitignore** **کارش چیست و چرا .env** **هرگز نباید commit شود؟** الگوی فایل‌هایی که گیت نادیده می‌گیرد (فقط untracked ها). `.env` رازها را دارد؛ لو رفتنش یعنی rotate فوری کلید + پاکسازی تاریخچه با `filter-repo`.

۱۸. **stash چیست و چه فرقی با کامیت موقت دارد؟** stash تغییرات را به صورت پشته موقت کنار می‌گذارد بدون کامیت – برای مانورهای کوتاه (فیکس فوری، pull). کامیت موقت تاریخچه را شلوغ می‌کند؛ stash تمیزتر است ولی فراموش‌شدنی – کارهای بلند = شاخه.

۱۹. **CI چیست؟ (و GitHub Actions؟)** اجرای خودکار lint/build/test روی هر `push/PR`. در گیت‌هاب با فایل YAML در `github/workflows/` تعریف می‌شود؛ خروجی‌اش checks در PR است که معمولاً merge را قفل می‌کنند.

۲۰. اولین روز کار: **workflow گیت تیم را چطور می‌فهمی؟** می‌پرسی: main محافظت شده است؟ استراتژی merge چیست (squash)? قرارداد نام شاخه و کامیت؟ چند approval لازم است؟ checks الزامی؟ – هشت سوال چک‌لیست فصل ۱۲.

۲۱. **گیت فایل‌ها را چطور ذخیره می‌کند؟** دیتابیس محتوا-محور با هش SHA: چهار آبجکت – blob (محتوای فایل، بدون اسم)، tree (پوشه)، commit (snapshot + والد + پیام) و tag. محتوای یکسان = blob مشترک (dedup)؛ شاخه فقط یک فایل متنی حاوی SHA است.

۲۲. **Git Hook چیست؟ چند نمونه؟** اسکریپتی که گیت در نقاط چرخه اجرا می‌کند؛ exit غیر صفر = لغو عملیات. **pre-commit** (چک محتوا مثل env. lint)، **commit-msg** (اعتبارسنجی پیام Conventional)، **pre-push** (تست قبل از ارسال). هوک‌ها در **.git/hooks** نسخه‌بندی نمی‌شوند – برای تیم: **core.hooksPath** یا Husky؛ و چون قابل رد شدن با **--no-verify** است، اعمال اجباری در CI تکرار می‌شود.

۲۳. **چطور پیدا کنی کدام کامیت یک رشته را وارد کد کرد؟** (pickaxe) **git log -S "apiKey"** کامیت‌هایی را نشان می‌دهد که تعداد آن رشته را تغییر داده‌اند؛ **-G** با regex. اگر عامل باگ را نمی‌دانی ولی علامتش را می‌دانی: **git bisect** (جست‌وجوی دودویی). برای «این خط را کی نوشته؟»: **git blame** یا GitLens.

۲۴. **git gc چه می‌کند و چرا مخزن با هزار کامیت کوچک می‌ماند؟** آبجکت‌های تکی (loose) را بسته‌بندی و فشرده می‌کند (delta compression + packfile) و آبجکت‌های بی‌مرجع را بعد از مهلت reflog پاک می‌کند. به‌خاطر dedup و delta، هر کامیت فقط «تغییرها» را واقعاً ذخیره می‌کند – snapshot بودن با هزینه دیسک کم سازگار است.

۱۶.۵ – نقشه راه پس از این دوره



سه توصیه پایانی: 1. چرخه PR را روی پروژه‌های خودت عادی کن – شاخه، کامیت مرتب، PR با قالب، squash merge. وقتی این چرخه برایت «کار روزمره» شد، در شرکت تازه‌کار نیستی. 2. زمین تمرین را نابود کن – هر هفته یک سناریوی فصل ۱۳ را عمداً بساز و نجات بده. ترس فقط با تجربه حادثه می‌میرد. 3. **داکیومنت رسمی را دوست بدار – git-scm.com/book** (کتاب رسمی Pro Git، رایگان) و **docs.github.com** همیشه به‌روزترین منبع‌اند؛ جزئیات نسخه‌ای این دوره (مثل Git 3.0 در راه) را با آن‌ها چک کن.

موفق باشی – از «کامیت و پوش» تا «آماده تیم»، مسیر رو کامل اومدی. 🚀